

CHARLESTON SOUTHERN UNIVERSITY

With It

COIN 499, SENIOR PROJECT

Sean Timothy Hayes

Dr. Valerie Sessions, Project Advisor

December 06, 2008

The program titled "With It" is a pitch and rhythm recognition and evaluation tool for musicians. It is written in Java and utilizes the MARF and *JMusic* libraries under the GPL license.

Contents

Introduction	2
Project Summary	2
Project Background	2
Project Purpose and Objectives	2
Research.....	3
Music and Frequency.....	3
Signal Processing to Determine Frequency.....	4
Determining the Fundamental Frequency	5
Frequency to Pitch.....	5
MIDI Format.....	5
Development.....	6
Proposed Schedule.....	6
Development Software	6
Language and Libraries	6
Design.....	6
Features and Layout.....	6
Icons and Logos.....	11
Problems and Solutions	12
Accurate Results from FFT	12
Determining the Fundamental	12
Displaying Music.....	13
Current Limitations.....	13
Code Highlights.....	13
Multithreaded.....	13
Pulse Event and Listener	13
Basic MIDI Playback.....	13
Overall Program Highlights.....	14
Appendix A: Correspondents	15
Message to Andrew Brown, a developer of <i>JMusic</i>	15
Discussion with Serguei A. Mokhov, creator of <i>MARF</i>	15

Introduction

Project Summary

This is a solo music analysis tool use by instrumentalists to improve and enhance their performance skill. The user is able to play along with the program which analyzes the notes of the instrument via a microphone. As the program analyzes the sound in real time, it determines the current pitch, as well as any variations in the intonation, measured in hertz. It also determines basic rhythms played by the musician. The user can load a MIDI file containing a solo and then play it while viewing the music on the screen. As the musician plays, the sound is captured, processed, and displayed in real time. The user can subsequently see how accurately he or she performed the piece. To assist the musician, a digital tuner, as well as a metronome, may be employed as part of the program and used simultaneously with the other features. In the future, the program will display a score based on pitch, intonation, and rhythm accuracy.

Project Background

Growth as a musician relies on a strong attention to detail. As musicians refine their skill, they become aware of more and more subtleties in the music and in their playing. Two constant areas of refinement are pitch and rhythm. An instrument's intonation is affected by temperature, humidity, design and many other factors. Also, fingerings, embouchure, and physical differences among people affect the pitch of individual notes on a single instrument. Therefore, musicians at all levels continue to work on their pitch accuracy. Likewise, musicians are continuously developing their sense of rhythm. In western music, rhythm is considered to be as continuous and constant as the movement of time. It is the goal of every musician to be able to accurately fit the rhythms into the ever present pulse that remains in time.

To accomplish these goals, musicians tediously break music down into its fundamental elements and repetitively practice them. Then the musician brings the music back together incorporating more and more elements. Another method is to practice all the elements simultaneously, but at an extremely slow tempo. As the musician develops confidence, the tempo is slowly increased. The danger in either method is that objectivity will be lost through reinforcement of progressively inaccurate repetition. It is easy to play a specific note slightly out of tune. Initially, this mistake may be obvious to a musician. However, by hearing this small error consistently and frequently, the error starts to sound correct. In fact, when the note is again played in tune, it may sound like an error. This is a slippery slope down which musicians can fall. A musician may leave the practice room confident in their ability to play a part only to be rudely awakened when integrating it with the other musicians. Likewise, rhythm is even more important when the musician plans to integrate his or her part with others.

Project Purpose and Objectives

With It provides a reliable measurement tool to monitor accuracy in practice. The metronome provides a consistent pulse at any tempo so a musician can compare their rhythms to.



The tuner provides a real time graphical representation of the musician's fundamental pitch. The main feature of the program allows the user to see his or her accuracy on a piece as a whole. Musicians can load their music in *With It* through the standard and widely available MIDI format. The musician can then play the piece as they view it on in the program's main window. The audio captured as the musician plays is processed and displayed in a comparable format. Musicians can become confident in the precision of their pitch and rhythms during practice. This greatly speeds up the learning progress as accuracy is continually reinforced.

Research

Music and Frequency

Sound travels in compression waves. Frequency is the number of waves that pass over a certain time: commonly measured in hertz (Hz), or waves per second. Rhythm and pitch are fundamentally combined in western music notation. A note's shape represents its duration while its vertical location represents its pitch. Every note consists of a single pitch or frequency.

However, when any instrument, such as the human voice or piano, produces a pitch it is actually producing many pitches simultaneously. The lowest frequency is the one the human ear hears and is called the fundamental pitch or fundamental frequency. All the other frequencies are multiples of the fundamental and are called overtones or harmonics. For example, the standard tuning pitch

for orchestra is the note *A* which is equivalent to the frequency 440Hz. When an instrument plays an *A* it will actually produce pitches at 440Hz, 880Hz, 1320Hz, and beyond. The presence and strength of these overtones shapes our perception of an instrument's tone and is what makes different types of instruments distinguishable. There is much speculation as to how the human brain determines the fundamental pitch. In many cases, the fundamental is not actually produced by the instrument. Yet, it is clearly heard by an audience. This makes the process difficult for computer analysis. For more information on this topic, please read *Determining the Fundamental Frequency* under the *Research* section. In Figure 2 below, harmonics are notated on a musical scale with sharps and flats colored coded.

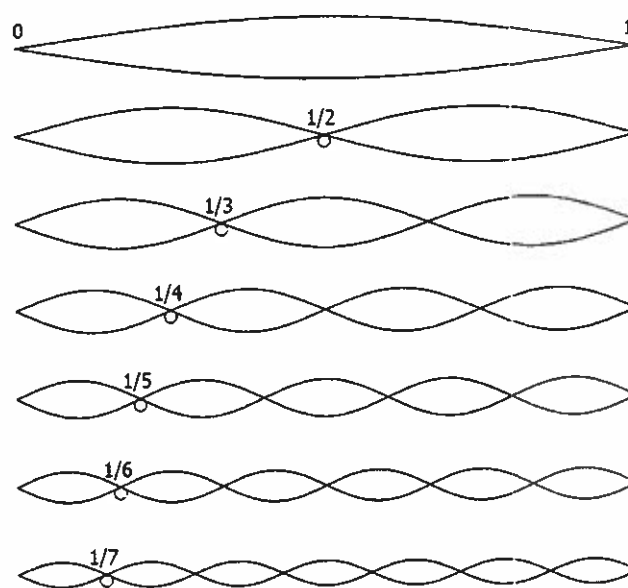


Figure 1: The overtone series shown in a vibrating string



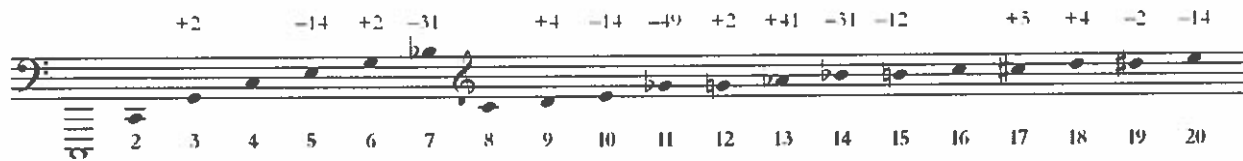


Figure 2: The harmonic series as musical notation. The numbers above the harmonic indicate the number of cents it deviates from equal temperament. Cents is a measurement of intonation in which one cent is one hundredth of a semitone. Red notes are sharp. Blue notes are flat

Signal Processing to Determine Frequency

There are many algorithms for determining frequency from captured sound data. These algorithms can be divided into two groups: time domain algorithms and frequency domain algorithms. The time domain algorithms are much easier to implement as they simply measure the distance between zero crossing points of the wave. However, this method does not work on complex waveforms with multiple sine waves. As explained earlier, instruments produce numerous waves simultaneously; consequently this method is not sufficient for this application.

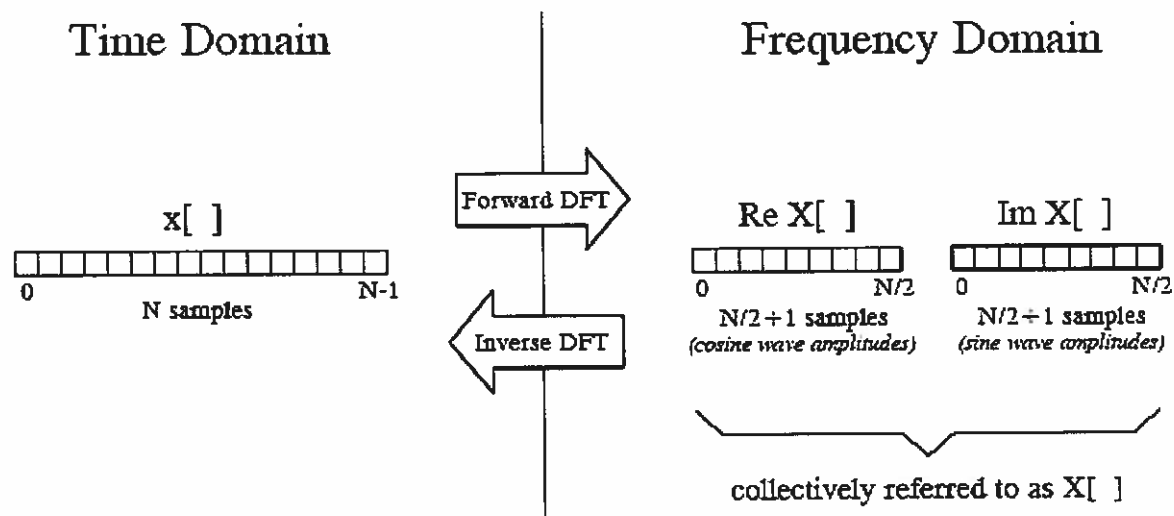


Figure 3: Conversion from Time Domain to Frequency Domain (from *The Scientist and Engineer's Guide to Digital Signal Processing* by Steven W. Smith)

Frequency domain approaches convolute or convert the wave data from the time domain into the frequency domain. As a result these algorithms require much more processor power. There are several approaches to this, but one of the oldest and most commonly used is the Fourier Transform. The Fast Fourier Transform is used in this program and has a runtime of $O(N \log N)$. It accepts an array of wave data with n elements where n is a power of 2 and outputs two arrays of length $n/2$ as seen in Figure 3. One array contains the real portion of the transform and the other the imaginary. Each element represents an interval of frequencies. The value in the element is a measure of amplitude. The larger the number, the higher the volume at that particular frequency range. Naturally, the larger the array, the more precise the frequency measurements are. The frequencies range from 0Hz to half the sample rate the sound was



captured at. This is based on the Nyquist–Shannon sampling theorem, which states that perfect reconstruction of a signal is possible when the sampling frequency is greater than twice the maximum frequency of the signal being sampled.

In order to determine the true magnitude of the real and corresponding imaginary, elements are combined using a sum of squares method: $|X_i| = \sqrt{(X_{real_i}^2 + X_{imaginary_i}^2)}$ where X is the magnitude. The Fast Fourier Transform method used by *With It* performs the step as a part of the computational process. From the resultant data the most prominent frequency ranges can be extrapolated.

Determining the Fundamental Frequency

A flawless method for determining the fundamental pitch from the frequency spectrum has yet to be discovered. There has been much research performed and many doctoral papers written on the subject; most of which is beyond my current mathematical comprehension. For more information read *Determining the Fundamental in Problems and Solutions* on page 12. For the purposes of this program the aforementioned process was employed.

Frequency to Pitch

Once the fundamental frequency is determined it is relatively easy to convert it to pitch. The formula is $P(f) = P_{ref} + O \log_2 \frac{f}{F_{ref}}$ where P_{ref} is a constant. The pitch number associated with the reference frequency F_{ref} . P_{ref} is usually 69 which is the MIDI number for A3, which is the third lowest A on a standard size keyboard. In America the equivalent frequency, F_{ref} is generally 440Hz; though it is commonly slightly higher in Europe. O is the number of notes in one octave: 12 in western music.

MIDI Format

MIDI or Musical Instrument Digital Interface is a long held standard for digital music. MIDI does not encapsulate a real audio signal, but transmits and stores it in event messages. Every pitch in the western chromatic scale has a pitch number. *With It* displays and stores its pitch data in ways consistent with the MIDI format. Also, the program utilizes MIDI files to display and compare the expected music and the audio processed from the microphone. The MIDI events are generated on the fly and sent to the sound card from the metronome. Some of the MIDI events utilized in these processes are note on, note off, tempo, time signature, key signature, and duration. The *JMusic* MIDI file classes did not fully implement all the MIDI events. As a result, the key signature and time signature events were implemented. Hopefully, these updates made for *With It* will be incorporated in future releases of the *JMusic* library (see Message to Andrew Brown, a developer of *JMusic* on page 15).



Development

Proposed Schedule

This was the original schedule that was made after Java was chosen as the programming language. This schedule was not closely followed as it took almost a full semester to get meaningful results from the Fast Fourier Transform microphone input. Also, other unforeseen research was necessary in the area of fundamental pitch extrapolation.

- October 8, 2007 – Researched Sound Analysis and Available Software.
- October 22, 2007 – Chosen Sound Libraries to Use.
- November 5, 2007 – Have a General Layout of the Project.
- November 22, 2007 – Research *MARF* libraries / Explore other options.
- March 29, 2008 – Produce a Program that Displays Recorded Pitches (a tuner).
- April 1, 2008 – Have a UML Diagram of the Entire Project.
- May 5, 2008 – Implement Rhythm and Note Detection.
- September 1, 2008 – Implement Feedback Including a Scoring Algorithm, and the GUI.
- November 3, 2008 – Have Documentation Completed.

Development Software

The program was written mostly using Netbeans 6.1 and 6.5. Later, version control was added using Subversion. Other programs used were Notepad++ and the GIMP. Documentation was created using Microsoft Word 2007 and JavaDoc. The program was developed and tested in Windows Vista and Ubuntu 8.4 and 8.10.

Language and Libraries

Library options were considered in conjunction with the programming language. The purpose was to find libraries that would allow the programmer to focus on uniqueness of the project rather than the details of acquiring sound from the microphone, reading MIDI files, and displaying notation. However, this goal was not met due to some unforeseen limitations in some of the libraries used. Considerable time was spent modifying the *JMusic* library to read MIDI files and display the music in standard notation. Another goal was portability. Java was the best fit with its extensive well documented standard libraries as well as the availability of the *JMusic* and *MARF* (*Modular Audio Recognition Framework*) libraries under the GPL. C++ with PortAudio was considered as it is designed for portability.

Design

Features and Layout

With It was designed and implemented as a multi-document application. There is no foreseeable limit to the number of files and recordings that can be open. When the program first starts, there is a spacious open area in which music and other features to be viewed (see Figure



4). As a portable application, *With It* takes on the look and feel of the operating system it is being run on/under (see Figure 5). If this is not possible, the standard Java swing look is used. The program has three main features: solo part analyzer, tuner, and metronome. Each of these can work concurrently. All these features are accessible from the menu bar as well as the main toolbar.

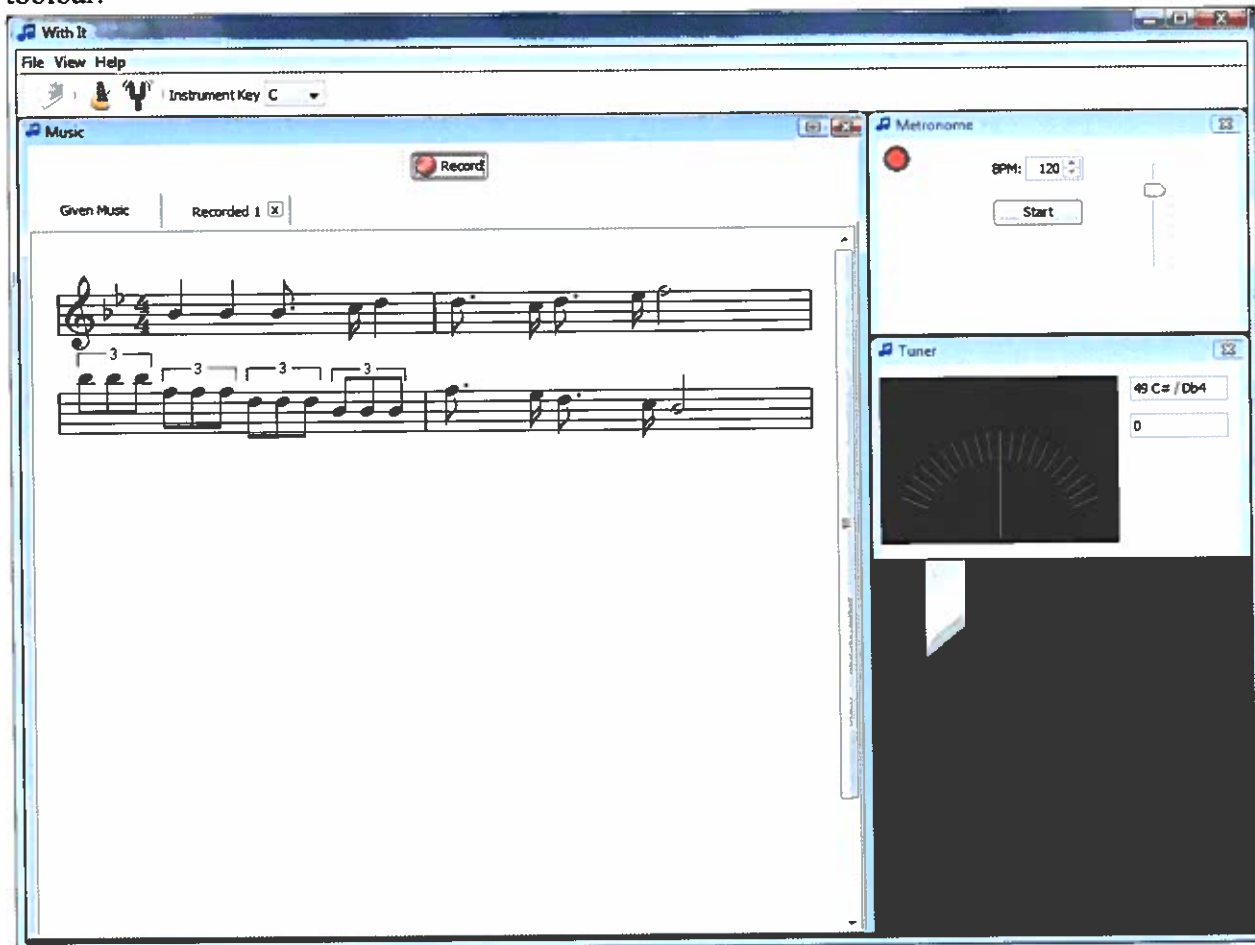


Figure 4: *With It* analyzing music, tuning, and providing pulse from the metronome simultaneously in Windows Vista.



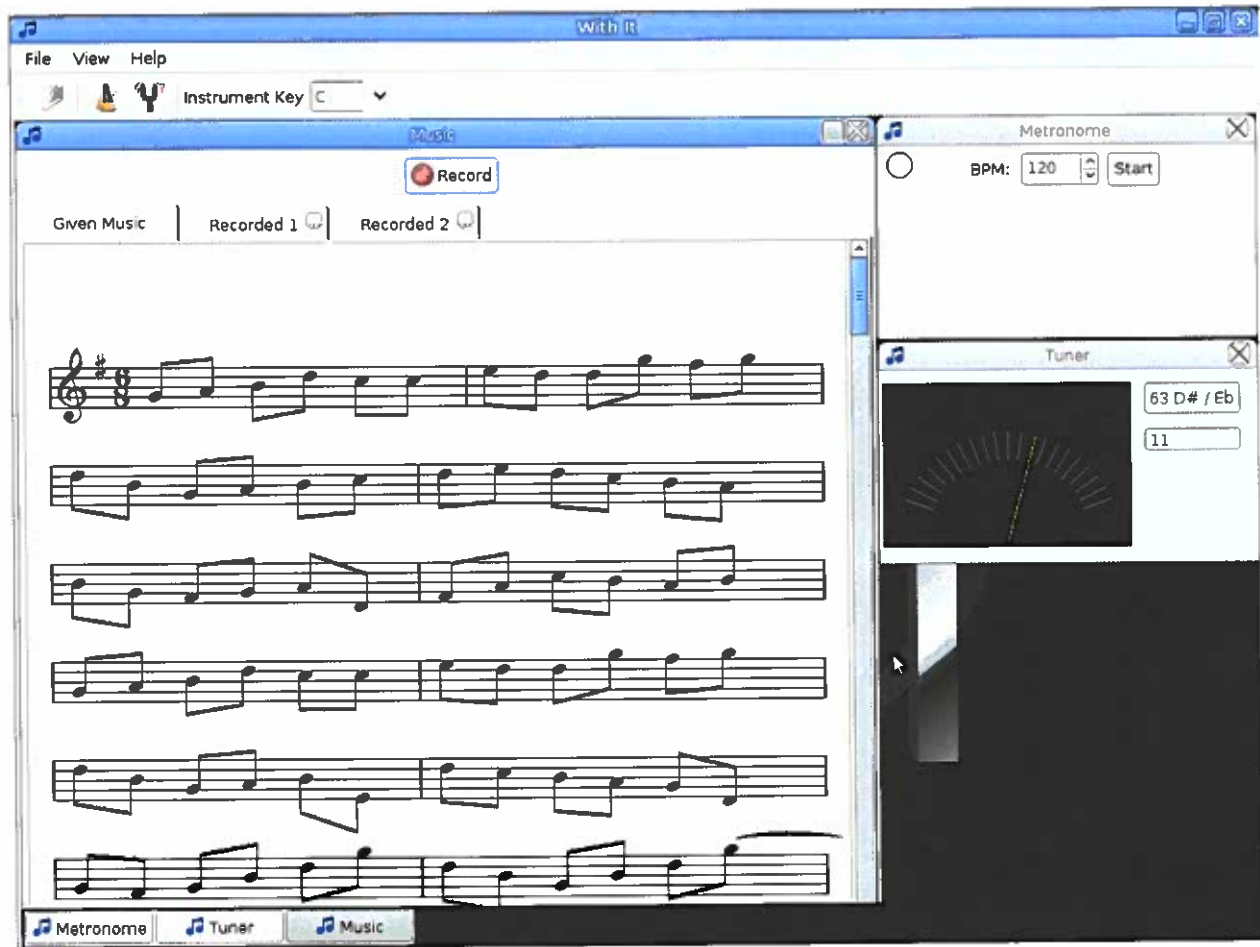


Figure 5: *With It* running in Gnome in the Ubuntu Operating System.

The tuner displays the note name and octave of the note to be tuned. The displayed note and octave are transposed to the instrument key for the convenience of the user. The tuner also displays how many cents flat or sharp the input pitch is. Cents are a measurement of intonation where one cent is one hundredth of a semitone. It is assumed that the user is tuning to the nearest semitone so the cents displayed will never be larger than ± 50 . The tuner also contains a graphical representation of how close the pitch is to the desired note. When the needle is completely vertical the pitch is completely in tune and the needle is green. The graphical tuner is scalable, growing and shrinking as the frame is resized.



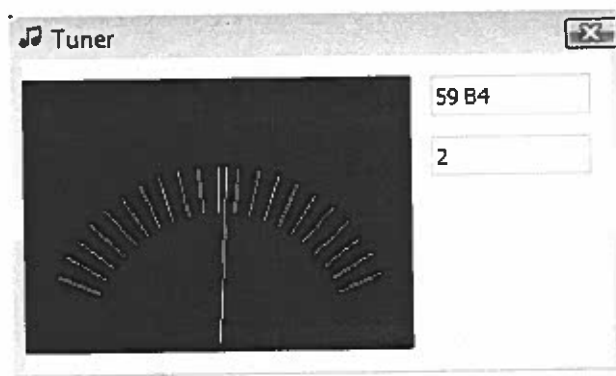


Figure 6: The tuner when the user is playing a B4 that is a mere 2 cents sharp.

The metronome provides a steady click for musicians to follow. This pulse is both visible and audible. The volume can be adjusted from muted to very loud using the vertical slider. The rate of this pulse is measured in beats per minute and can be changed by a combo box. The metronome can also be started and stopped as necessary.

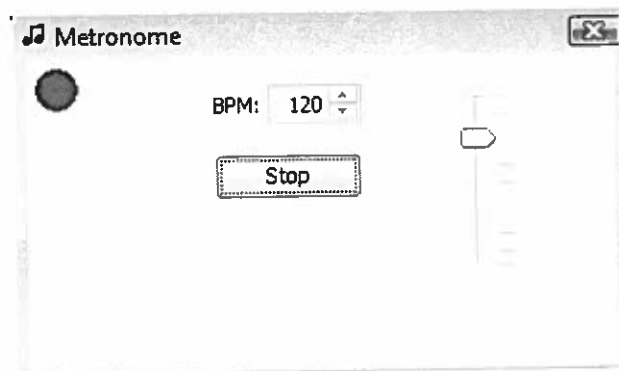
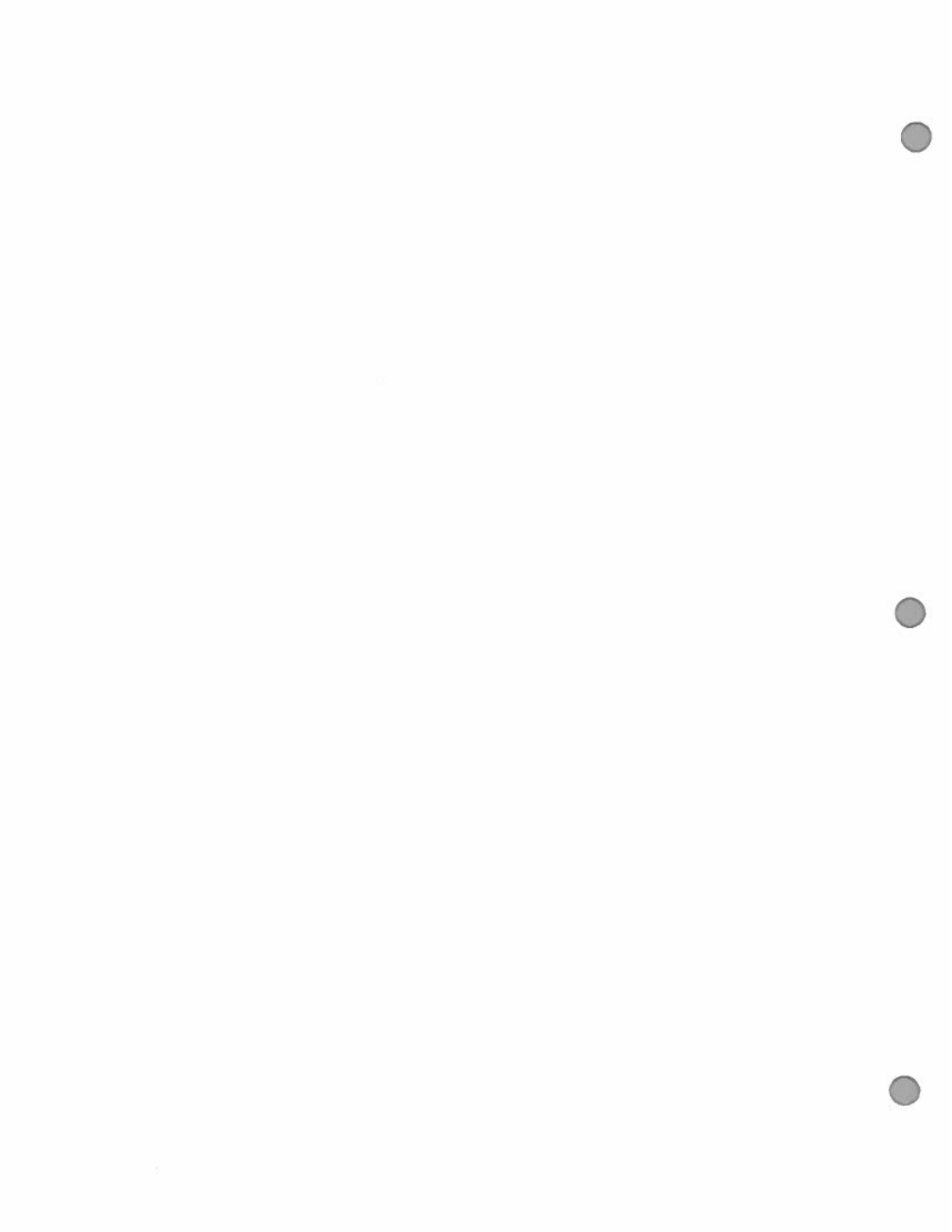


Figure 7: The metronome running at 120 beats per minute.

The main tool is the music analyzer. This tool is accessed by opening a MIDI file containing a solo to play. The solo is loaded in a new window so the user can read it while he or she plays. When the user presses record a one measure count off is given and a new tab is added to the window. After the one measure prep the program begins capturing input from the microphone. The metronome may remain on to continue to provide the user with a steady beat. As the user plays, the new tab is updated to contain the notes that the user actually played. Recording stops when the user deselects the record button. The user has the option to close any tabs that have been created from recordings. The user can also open as many music file at a time as is desired. Both the MIDI and captured music staves are transposed to the key used by the musician's instrument.



The screenshot shows a software window titled "Music" with a toolbar containing a "Record" button and a close button. Below the toolbar are four tabs: "Given Music", "Recorded 1", "Recorded 2", and "Recorded 3". The "Recorded 3" tab is selected. The main area displays musical notation for "Row, Row, Row Your Boat" in B-flat major. The notation consists of two staves. The first staff shows the melody in 4/4 time, and the second staff shows a bass line with triplets. The "Recorded 3" tab is active, indicating the third analysis is still being recorded.

Figure 8: The Music Analysis Window showing *Row, Row, Row Your Boat* in B flat major with three analyses of the captured solos. The third is still being recorded.

The main toolbar gives the user quick access to commonly used features. It also allows the user to select the key of his or her instrument. Most woodwind instruments sound at a different key than they play. This allows for easier reading as the music for these instruments is adjusted so that most of the notes will be in the middle of the staff and few ledger lines will be needed. However, as a result, the sheet music must be altered for each instrument.





Figure 9: The Main Toolbar where an E flat instrument is being selected. Such instruments include Alto and Baritone Saxophone.

Icons and Logos

The program is titled *With It* to emphasize the ability that it has to determine how closely one follows the music. The logo was created by Carolyn Mills and represents the short hand for with (w/) followed by “it.” The “/” is doubles as a musical note.

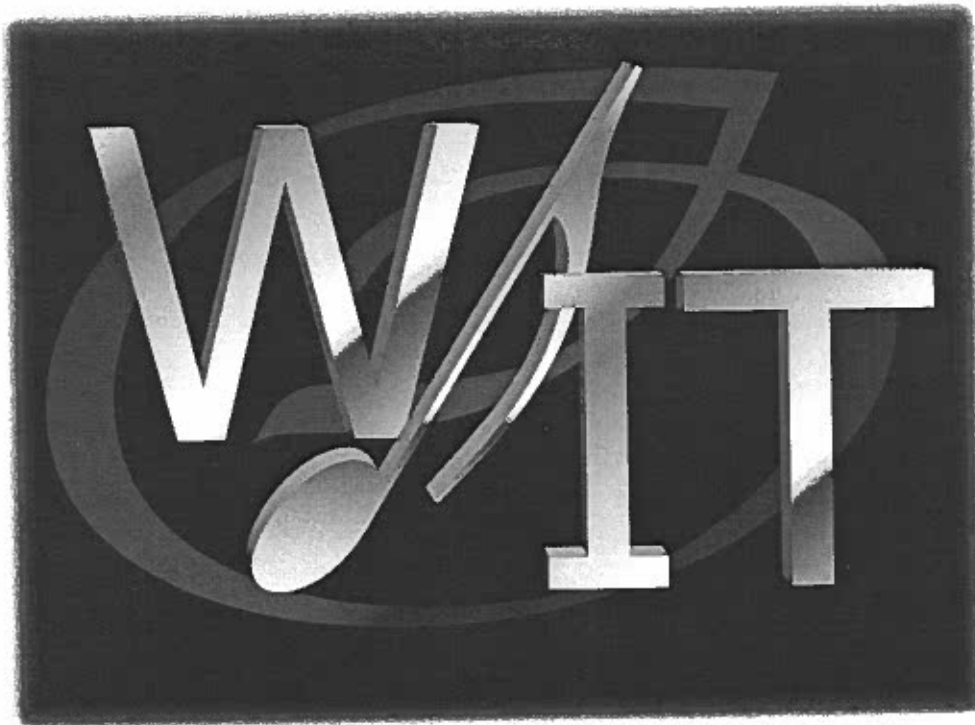


Figure 10: The *With It* logo

Some of the icons could be used from the Java library. However, no free icons could be found for the tuner or metronome. The metronome icon was created by Sean Hayes and the tuner icon by Erin Hayes (see Figure 9). Both were created in the GIMP.

Problems and Solutions

Accurate Results from FFT

Many different implementations were tested before settling on the one in the *MARF* library. However, it wasn't until several months of work had passed before it was discovered that input from the microphone had to be altered before feeding it to the FFT. In Java, sound is captured as a byte stream. The stream must be divided into chunks twice the size that is to be fed into the FFT. Then every two bytes must be combined and scaled to values between -1 and 1. Apparently, audio from wave files is already scaled in this manner.

Determining the Fundamental

As described in *Music and Frequency* under *Research* on page 3, is difficult to determine the fundamental frequency, even given the frequency spectrum. The simplest and most widely circulated method would be to assume that the frequency range with the largest amplitude or magnitude would be the fundamental. However, in my tests I discovered that this is usually not the case.

Another observation made was that in a moderately quiet room the top six frequency ranges would generally be overtones or something in the lowest frequency ranges X_0 or slightly above. This was true even with a loud metronome producing short regular beeps in the background. By ignoring frequencies below the lowest piano note and using the top four to ten frequency ranges, consistent overtones can be found. The second method that was attempted was to merely use the lowest of these values. This produced more accurate results, but still about half the notes were registering as the first or second overtone rather than the actual fundamental. This meant that the fundamental was consistently very weak. These tests were completed using piano, saxophone, harmonica, and humming.

In search for an even better method, it was reasoned that one could simply find the greatest common factor (GCF) of the overtones if any odd number overtone was present the fundamental would be found. However, since we are dealing with frequency ranges rather than actual frequencies an exact GCF would not produce the fundamental. Another problem was the need to find the greatest common factor for all the top frequencies. By allowing error in the GCF the result of comparing two frequency ranges may produce the incorrect fundamental and therefore propagate the error through the rest of the comparisons. A pseudo-GCF algorithm was developed under the assumption that any pitch observed would be within 5 multiples of the fundamental and provides a customizable acceptable error range.

The current method used is a combination of the last two methods. First, each probable overtone is compared with every other probable overtone producing n^2 results where n is the number of probable overtones. Then the lowest resulting value is assumed to be the fundamental. This produces accurate results about 90% of the time. A simple and reliable method is still being searched for.



Displaying Music

JMusic provides a series of classes for displaying music in various formats. However, the options for displaying music in standard notation are very limiting. Also, the library hasn't been updated since 2004 and it does not work with the light weight swing elements which are preferred in this application. As a result it seemed necessary to make my own. I discovered a program called *Impro-Visor* which also used a version of *JMusic*, but it had its own display system which is severely modified from *JMusic*. Many components of *Impro-Visor* were modified and integrated into *With It*. This was a difficult process as the display was heavily integrated into *Impro-Visor*.

Current Limitations

- No volume control
- Audio buffer does not flush after each capture
- Preferences are not saved
- Comparison with the MIDI music is not implemented
- Metronome visual does not sync-up with analyzer
- No control over metronome when analyzing
- Tuner must be off before the first analysis of each piece
- Analysis is only about 90% accurate
- In Gnome exit buttons on tabs are too small to display the 'x'.

Code Highlights

Multithreaded

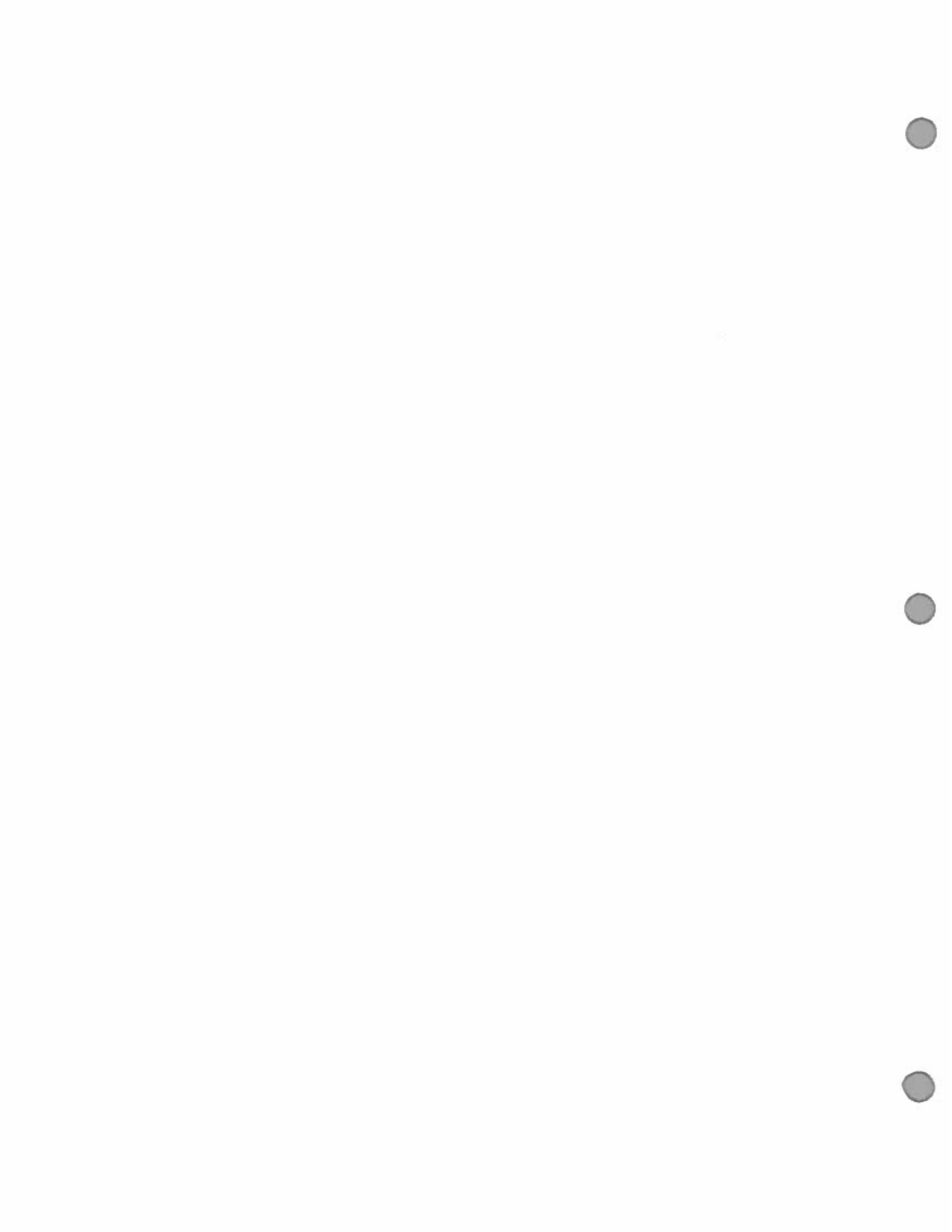
It is necessary to use threads extensively in this application. Threads are utilized in about 25 different classes.

Pulse Event and Listener

It is necessary for certain program elements to identify when a beat of the metronome has occurred. To allow this to happen and metronome produces a *PulseEvent* every beat. Object can then implement the *PulseListener* to perform specialized tasks every time a pulse occurs.

Basic MIDI Playback

Metronomes are generally audible and it was therefore necessary provide a way to produce sound. Through the use of the Java sound API the metronome retrieves a synthesizer and opens a channel for the audio data. When the metronome is started, a thread is created that sends note on and note off signals to the sound card through the open channel.



Overall Program Highlights

Though there are areas yet to be developed and debugged for this program to run optimally, it can be used as describe (in this prototype) to assist musicians in their practice. It is exciting to work on the human-computer interaction for music performance and I hope to continue to develop this program.



Figure 11: Andrew Felker, a senior music major, using *With It* while practicing Xylophone

Appendix A: Correspondents

Message to Andrew Brown, a developer of *JMusic*

Sean Hayes <stretchtiberius@gmail.com>

Tue, Dec 2, 2008 at 11:35 AM

To: a.brown@qut.com

I've been using *JMusic* for a project I'm working on and really appreciate your work. However, I needed to be able to read the time signature and key signature events from a MIDI file. I noticed that these were not fully implemented so I took the liberty of adding some code.

Attached are the files I modified. I hope that they are useful for future releases of *JMusic*.

Sincerely,
Sean Hayes

 JM_Mod.zip
14K

Andrew Brown <a.brown@qut.edu.au>

Tue, Dec 2, 2008 at 6:36 PM

To: Sean Hayes <stretchtiberius@gmail.com>

Hi Sean,

Thanks very much for these additions to jMusic. Much appreciated.

I hope you project with jMusic went well.

Cheers,

Andrew

Discussion with Serguei A. Mokhov, creator of *MARF*

how to set iRequiredFrequency

By: Nobody/Anonymous (nobody) - 2008-08-21 01:07

I am somewhat new to audio analysis so I apologies if this is a silly question. I am trying to use MARF's FFT support to determine the frequencies of musical instruments. The results I am getting are not very accurate. I think that I may need to sample at a higher frequency in order to get better results. However, there doesn't seem to be a way to change iRequiredFrequency which is set to its default (80000Hz).

Thanks for your help.



RE: how to set iRequiredFrequency

By: StretchTiberius ([stretchtiberius](#)) - 2008-08-21 14:17

Just to make myself more clear. I am referring to iRequiredFrequency - Variable in class marf.Storage.SampleLoader (Current frequency).

RE: how to set iRequiredFrequency

By: Serguei A. Mokhov ([mokhov](#)  ) - 2008-08-21 15:17

Hi,

> I am somewhat new to audio analysis so I apologies
> if this is a silly question. I am trying to use MARF's
> FFT support to determine the frequencies of musical
> instruments. The results I am getting are not
> very accurate.

I am just curious how did you set up your experiments?
Are going through the pipeline in the MARF's class or
are you invoking the classes you need directly?

What kind of results are getting, i.e. "not very accurate"?

What kind of samples (WAV files) are you using for training
and testing (i.e. encoding details).

It's in general good to know of the environment. Also
the MARF version you are using.

> I think that I may need to sample at a higher frequency
> in order to get better results.

This may certainly help. The default is 8000 Hz (not 80000),
which is roughly corresponding to the quality of the phone
line; if you have a higher quality data, you must definitively
set a more compatible sampling frequency.

> However, there doesn't seem to be a way to change
> iRequiredFrequency which is set to its default (80000Hz).

The SampleLoader class has the standard get/setRequiredFrequency()
methods... that brings me back to the question which MARF version



you are using, in particular the revision of the SampleLoader class (found in the @version Javadoc comment on top of the class).

This is the latest current revision of the class so far for comparison:

http://marf.cvs.sourceforge.net/*checkout*/marf/marf/src/marf/Storage/SampleLoader.java
<http://marf.sourceforge.net/docs/marf/0.3.0.6/api-dev/marf/Storage/SampleLoader.html>

Thanks,

-s

RE: how to set iRequiredFrequency

By: StretchTiberius ([stretchtiberius](#)) - 2008-08-22 16:06

>I am just curious how did you set up your experiments?
>Are going through the pipeline in the MARF's class or
>are you invoking the classes you need directly?

I am just invoking the classes I need directly.
i.e.:

```
File oImportWaveFile = new File("C523.25sop.wav");  
WAVLoader wavFile = new WAVLoader();  
// wavFile.setRequiredFrequency( 22000 );  
// MarfTest.java:32: cannot find symbol  
//symbol : method setRequiredFrequency(int)  
Sample wavSample = wavFile.loadSample(oImportWaveFile);
```

I am also calling the FFT function directly. It just seemed easier. Is this not recommended?

>What kind of results are getting, i.e. "not very accurate"?

I've made some progress with this since my last post. Right now it seems that the majority of the frequencies are just under double what they should be.

i.e. Input at 440Hz is output at 877.8225314292689Hz
Input at 523.25Hz is output at 1045.2825582814598Hz
I have yet to test a larger range of frequencies.
Could this be Aliasing (again I'm new to this)?



>What kind of samples (WAV files) are you using for training
>and testing (i.e. encoding details).

The wav files are mono 16bit 8000Hz. However, I would record these samples at a higher sample rate. Soon I want to do this take input directly from the microphone.

>It's in general good to know of the environment. Also
>the MARF version you are using.

I was originally using marf-0.3.0-devel-20060226. Today I tried 0.3.0.6-snapshot-0.3.0-devel-20070108 with the same results. I searched the website for a more recent jar file/source package. Where can I find this?

>The SampleLoader class has the standard get/setRequiredFrequency()

I guess that was added after the 20070108 snapshot.

I really appreciate your help. Let me know if you would like any more information.

RE: how to set iRequiredFrequency

By: Serguei A. Mokhov ([mokhov](#)) - 2008-08-27 16:52

```
>>I am just curious how did you set up your experiments?  
>>Are going through the pipeline in the MARF's class or  
>>are you invoking the classes you need directly?  
>  
>I am just invoking the classes I need directly.  
>i.e.:  
>  
>File oImportWaveFile = new File("C523.25sop.wav");  
>WAVLoader wavFile = new WAVLoader();  
>// wavFile.setRequiredFrequency( 22000 );  
>// MarfTest.java:32: cannot find symbol  
>//symbol : method setRequiredFrequency(int)  
>Sample wavSample = wavFile.loadSample(oImportWaveFile);  
>  
>I am also calling the FFT function directly. It just seemed easier.  
>Is this not recommended?
```

No that's alright. The reason I asked if you were going through the pipeline in the current way of things it would be more



difficult to set other-than-the-default sampling frequency :)

```
>>What kind of results are getting, i.e. "not very accurate"?  
>  
> I've made some progress with this since my last post.  
> Right now it seems that the majority of the frequencies  
> are just under double what they should be.  
> i.e. Input at 440Hz is output at 877.8225314292689Hz  
> Input at 523.25Hz is output at 1045.2825582814598Hz  
> I have yet to test a larger range of frequencies.  
> Could this be Aliasing (again I'm new to this)?
```

Hmmm interesting. It's hard to tell at this point for me without seeing what you are actually doing, but in deference to your later post as well as the fact that you should know that the FFT analysis does windowing, so this cuts things in half -- i.e. we are really doing 1024 samples, but get 512 because of windowing and so on, so you maybe observing the opposite of that, but I will need to do more tracing to see why things behave like that and the way you invoke stuff after you loaded the samples.

```
>>It's in general good to know of the environment. Also  
>>the MARF version you are using.  
>  
>I was originally using marf-0.3.0-devel-20060226.  
>Today I tried 0.3.0.6-snapshot-0.3.0-devel-20070108  
>with the same results. I searched the website for a more  
>resent jar file/source package. Where can I find this?  
>  
>>The SampleLoader class has the standard get/setRequiredFrequency()  
>  
>I guess that was added after the 20070108 snapshot.
```

Shucks... I double checked and indeed somehow these escaped the latest snapshot. I've been using the revising the CVS version, but I guess now that more people are using the official file releases, another snapshot release is well overdue :(

Before I make a release of the new snapshot, would you please try with one of the .jar's in my school's page? I compiled the last one from the working checkout:

<http://users.encs.concordia.ca/~mokhov/marf/>

Do let me know if it helps (or doesn't).

Thanks!



- 8

RE: how to set iRequiredFrequency

By: StretchTiberius ([stretchtiberius](#)) - 2008-08-25 02:03

>>What kind of results are getting, i.e. "not very accurate"?

>I've made some progress with this since my last post. Right now it seems that the majority of the frequencies are just under >double what they should be.

>i.e. Input at 440Hz is output at 877.8225314292689Hz

>Input at 523.25Hz is output at 1045.2825582814598Hz

>I have yet to test a larger range of frequencies.

>Could this be Aliasing (again I'm new to this)?

Something interesting here is that the 438.91126571463445Hz is also registering just at a lesser amplitude. I guess the first overtone is stronger on my recording. The recording is actually me playing a soprano saxophone into my computer microphone. I used a tuner to make sure it was right on pitch. Here are the top 10 results from this experiment on a soprano sax playing A 440Hz.

1653.3783977981452 @ index 1798 Frequency = 877.8225314292689 Hz
1122.1303664160816 @ index 1799 Frequency = 878.3107530818992 Hz
1024.0565274509938 @ index 899 Frequency = 438.91126571463445 Hz
372.5350991981543 @ index 1797 Frequency = 877.3343097766386 Hz
372.5350991981543 @ index 1797 Frequency = 877.3343097766386 Hz
287.6803161169527 @ index 1800 Frequency = 878.7989747345295 Hz
252.98917162807265 @ index 2697 Frequency = 1316.7337971439033 Hz
219.22603277261533 @ index 2698 Frequency = 1317.2220187965336 Hz
159.85865239224793 @ index 3597 Frequency = 1756.133284511168 Hz
147.68668321697962 @ index 3596 Frequency = 1755.6450628585378 Hz

RE: how to set iRequiredFrequency

By: Serguei A. Mokhov ([mokhov](#)  ) - 2008-08-27 17:24

I would be curious to see how you getting your results here. Also, something more obvious struck me here that I forgot to ask you: do you do any normalization and filtering (like noise/silence)? It is important in general for the sample data to be normalized by



amplitude [-1;1] prior doing FFT, and sometimes after doing the FFT. As a side note it also helps to remove any silence gaps as well as noise. For instance instantiating `marf.Preprocessing.FFTFilter.LowPasFilter` on your loaded sample, then invoking its methods `preprocess()`, `removeSilence()` (note having `LowPassFilter` itself implies noise removal of noise that is usually happening on high frequencies). `preprocess()` does call `normalize()` and `filter()` for you. Give it a try and see if this helps your results.

-s

RE: how to set iRequiredFrequency

By: StretchTiberius ([stretchtiberius](#)) - 2008-09-01 13:29

Thanks for all you help. I have not had much time to work on this lately because I am focusing on studying for the GRE. As soon as I can I will work through all this great information you gave me.

>I would be curious to see how you getting your results here.
I took the then greatest results from the FFT and calculated the frequencies as follows:
`HalfSampleRate = 4000;`
`HalfChunkSize = 2048;`
`frequency = ArrayIndex * HalfSampleRate / (HalfChunkSize + 1);`

>do you do any normalization and filtering (like noise/silence)?
I hadn't thought to do that. I work on adding the appropriate filters next.

Thanks again!



Senior Project

12/5/08 5:04 PM

Package Summary		Page
<u>imp</u>		59
<u>polya</u>		159
<u>sp</u>		25

Package sp

Interface Summary		Page
<u>AnalysisReceiver</u>	This interface should be used by any object that wishes to receive updated information from an Analyzer object.	26
<u>Buffer</u>	A simple FIFO buffer interface.	33
<u>Frequency</u>	Contains methods for extrapolating information based on a single frequency	48
<u>PulseListener</u>	The listener interface for receiving pulse events from the metronome.	58

Class Summary		Page
<u>Analyzer</u>	Obtains chunks of data from a buffer and extrapolates music.	28
<u>BlockingBuffer</u>	This is a thread safe FIFO buffer with 20 positions.	31
<u>Capture</u>	A class to capture chunks from the microphone input and send them to the specified objects (TunerPanel and/or Analyzer).	35
<u>Chunk</u>	Embodies on chunk of windowed audio data.	40
<u>CustomFileFilter</u>	This class allows for the creation of a file filter that can easily be edited during runtime.	46
<u>Metronome</u>	This class is the heart of rhythmic measurement and is used for any action that is required to happen on the beat.	50
<u>PlayPulseThread</u>	This class is used by the <code>Metronome</code> class to produce and even pulse in the specified tempo.	54
<u>PulseEvent</u>	This event occurs at every pulse of the metronome.	57

Interface AnalysisReceiver

sp

```
public interface AnalysisReceiver
```

This interface should be used by any object that wishes to receive updated information from an Analyzer object.

Author:

Sean Timothy Hayes

See Also:Analyzer, `sp.gui.MusicPanel`

Method Summary		Page
void	<u>addLatestColor</u> (<code>imp.gui.Staff.NoteColor color</code>) Sets the color for the latest note of the user phrase.	27
void	<u>addLatestNote</u> (<code>Note note</code>) Used to add the latest note to the user phrase.	26
void	<u>setExpectedPhrase</u> (<code>Phrase phrase</code>) Used to update the phrase storing captured solo from microphone	26
void	<u>setUserPhrase</u> (<code>Phrase phrase</code>) Used to update the given MIDI music	26

Method Detail

setUserPhrase

```
void setUserPhrase(Phrase phrase)
```

Used to update the given MIDI music

Parameters:

phrase - the given MIDI music

See Also:`sp.gui.MusicPanel.setUserPhrase(jm.music.data.Phrase)`**setExpectedPhrase**

```
void setExpectedPhrase(Phrase phrase)
```

Used to update the phrase storing captured solo from microphone

addLatestNote

```
void addLatestNote(Note note)
```

Used to add the latest note to the user phrase.

addLatestColor

```
void addLatestColor (imp.gui.Staff.NoteColor color)
```

Sets the color for the latest note of the user phrase.

Parameters:

`color` - the color the latest note should be

Class Analyzer

sp

```
java.lang.Object
└─ sp.Analyzer
```

```
public class Analyzer
extends Object
```

Obtains chunks of data from a buffer and extrapolates music. It then compares the generated music with the expected music presented in `expectedPhrase`

Author:

Sean Timothy Hayes

See Also:

[AnalysisReceiver](#)

Constructor Summary	Page
Analyzer (AudioFormat audioFormat, Buffer buffer, AnalysisReceiver receiver) Preferred Constructor	28

Method Summary	Page
void doAnalysis () Start retrieving chunks from the chunk buffer and analyze them.	30
int getBeatsPerMinute () Get Evaluation Tempo	29
Phrase getUserPhrase () Get the user phrase.	29
void setAnalysisReceiver (AnalysisReceiver receiver) Route the results to a different receiver	29
void setAudioFormat (AudioFormat audioFormat) Set the audio format	30
void setBeatsPerMinute (int bpm) Sets the tempo that the captured input should be evaluated in.	29
void setExpectedPhrase (Phrase phrase) Sets the phrase and the tempo based off of the expected phrase.	29

Constructor Detail

Analyzer

```
public Analyzer(AudioFormat audioFormat,
               Buffer buffer,
               AnalysisReceiver receiver)
```

Preferred Constructor

Parameters:

`buffer` - contains captured chunks from `capture`

Method Detail

setAnalysisReceiver

```
public final void setAnalysisReceiver(AnalysisReceiver receiver)
```

Route the results to a different receiver

Parameters:

receiver - the AnalysisReceiver that the data should be sent to.

setExpectedPhrase

```
public void setExpectedPhrase(Phrase phrase)
```

Sets the phrase and the tempo based off of the expected phrase.

Parameters:

phrase - the prase to be compared to

getUserPhrase

```
public Phrase getUserPhrase()
```

Get the user phrase.

Returns:

the phrase captured from the microphone.

setBeatsPerMinute

```
public void setBeatsPerMinute(int bpm)
```

Sets the tempo that the captured input should be evaluated in.

Parameters:

bpm - Beats Per Minute (must be greater than zero.

getBeatsPerMinute

```
public int getBeatsPerMinute()
```

Get Evaluation Tempo

Returns:

the evaluation tempo in Beats per Minute

setAudioFormat

```
public void setAudioFormat(AudioFormat audioFormat)
```

Set the audio format

Parameters:

`audioFormat` - the should be the same as the audio format used to capture the wave data.

doAnalysis

```
public void doAnalysis()
```

Start retrieving chunks from the chunk buffer and analyze them.

Class `BlockingBuffer`

sp

```
java.lang.Object
└─ sp.BlockingBuffer
```

All Implemented Interfaces:

[Buffer](#)

```
public class BlockingBuffer
extends Object
implements Buffer
```

This is a thread safe FIFO buffer with 20 positions. If all 20 positions become full, the next thread to call `set` will be asked to wait. This buffer stores objects of class `Chunk`.

Author:

Sean T. Hayes

See Also:

[Buffer](#), [Chunk](#)

Constructor Summary	Page
BlockingBuffer() Constructor	31

Method Summary	Page
void flush() Removes all chunks from the buffer.	32
Chunk get() Get a Chunk	32
boolean isEmpty() Find out if the buffer is empty.	32
void set (Chunk aChunk) Adds a Chunk to the end of the buffer	31

Constructor Detail

`BlockingBuffer`

```
public BlockingBuffer()
```

Constructor

Method Detail

`set`

```
public final void set(Chunk aChunk)
    throws InterruptedException
```

Adds a Chunk to the end of the buffer

Specified by:

set in interface Buffer

Parameters:

aChunk - chunk to add to the buffer

Throws:

InterruptedException

get

```
public final Chunk get()  
    throws InterruptedException
```

Get a Chunk

Specified by:

get in interface Buffer

Returns:

the chunk that has been in the buffer the longest

Throws:

InterruptedException

isEmpty

```
public final boolean isEmpty()
```

Find out if the buffer is empty.

Specified by:

isEmpty in interface Buffer

Returns:

true if the buffer contains on elements

flush

```
public void flush()
```

Removes all chunks from the buffer.

Specified by:

flush in interface Buffer

Interface Buffer

sp

All Known Implementing Classes:

[BlockingBuffer](#)

```
public interface Buffer
```

A simple FIFO buffer interface.

Author:

Sean T. Hayes

Method Summary		Page
void	<u>flush()</u> Removes all Elements from the buffer	34
Chunk	<u>get()</u>	33
boolean	<u>isEmpty()</u> Is the buffer empty	34
void	<u>set</u> (Chunk audio) Adds a another chunk to the end of the buffer	33

Method Detail

set

```
void set(Chunk audio)  
    throws InterruptedException
```

Adds a another chunk to the end of the buffer

Parameters:

audio - the chunk to be added

Throws:

InterruptedException

get

```
Chunk get()  
    throws InterruptedException
```

Returns:

the next oldest element in the buffer

Throws:

InterruptedException

isEmpty

`boolean isEmpty()`

Is the buffer empty

Returns:

true if the buffer has no elements in it currently

flush

`void flush()`

Removes all Elements from the buffer

Class Capture

sp

```
java.lang.Object
└─ sp.Capture
```

```
public class Capture
extends Object
```

A class to capture chunks from the microphone input and send them to the specified objects (TunerPanel and/or Analyzer). Capturing is done on one separate thread.

Author:

Sean T. Hayes

See Also:Chunk, sp.gui.TunerPanel, Analyzer

Constructor Summary	Page
Capture (<u>Buffer</u> buffer, sp.gui.TunerPanel tunerPanel, <u>AnalysisReceiver</u> analysisReceiver, AudioFormat audioFormat) Capture Constructor	36

Method Summary	Page
static double byteArrToDouble (byte[] arr, int start)	39
static int byteArrToInt (byte[] arr, int start) Found at http://www.captain.at/howto-java-convert-binary-data.php	39
float computeQuantizedSampleRate (float tempo) Calculates a sample rate that will result in producing chunks captured at a length of time equivalent to one beat of the given tempo.	37
Analyzer getAnalyzer ()	37
boolean getAnalyzerOn () See if the Analyzer is currently running	37
AudioFormat getAudioFormat () Get the audio formate to be captured	38
int getChunkSize () Gets Chunk Size	38
void setAnalysisReceiver (<u>AnalysisReceiver</u> receiver) Route the results to a different reliever	37
void setAnalyzerOn (boolean isOn) Option to send data to the Buffer to be used by the analyzer	36
void setAudioFormat (AudioFormat audioFormat) Set the audio formate for capturing.	39
void setBeatsPerMinute (int bpm) Set the tempo of the Analyzer	36
void setChunkSize (int chunkSize) Sets the size of the chunks to be captures.	37

void	<u>setQuantizedSampleRate</u> (float beatsPerMinute) Sets the sample rate that will result in producing chunks captured at a length of time equivalent to one beat of the given tempo.	38
void	<u>setSampleRate</u> (float sampleRate) Set the sample rate to be used for capturing.	38
void	<u>setTunerOn</u> (boolean isOn) Option to send data to the tuner	36

Constructor Detail

Capture

```
public Capture(Buffer buffer,
               sp.gui.TunerPanel tunerPanel,
               AnalysisReceiver analysisReceiver,
               AudioFormat audioFormat)
```

Capture Constructor

Parameters:

buffer - the buffer to which the captured chunks are stored
tunerPanel - the tunerPanel to receive Frequency Information
audioFormat - the audio format to capture audio in

See Also:

[Frequency](#)

Method Detail

setTunerOn

```
public void setTunerOn(boolean isOn)
```

Option to send data to the tuner

Parameters:

isOn - if true, chunk is sent to the tuner when captured

setAnalyzerOn

```
public void setAnalyzerOn(boolean isOn)
```

Option to send data to the Buffer to be used by the analyzer

Parameters:

isOn - if true, chunk is sent to the buffer when captured

setBeatsPerMinute

```
public void setBeatsPerMinute(int bpm)
```

Set the tempo of the Analyzer

Parameters:

bpm - Beats Per Minute (must be greater than zero).

getAnalyzerOn

```
public boolean getAnalyzerOn()
```

See if the Analyzer is currently running

Returns:

true if data is being stored in the buffer for an analyzer.

setAnalysisReceiver

```
public void setAnalysisReceiver(AnalysisReceiver receiver)
```

Route the results to a different receiver

Parameters:

receiver - the AnalysisReceiver that the data should be sent to.

getAnalyzer

```
public Analyzer getAnalyzer()
```

Returns:

the analyzer using this Capture object's output

setChunkSize

```
public void setChunkSize(int chunkSize)
```

Sets the size of the chunks to be captured.

Parameters:

chunkSize - must be a power of 2. If it isn't this function has no effect.

computeQuantizedSampleRate

```
public final float computeQuantizedSampleRate(float tempo)
```

Calculates a sample rate that will result in producing chunks captured at a length of time equivalent to one beat of the given tempo. This calculation is also related to the chunk size. A decrease chunk size will increase the resulting sample rate.

Parameters:

tempo - the tempo used in BeatsPerMinute. Note that if a division smaller than one beat is preferred then multiply the tempo by the subdivision.

Returns:

a sample rate to be used for tempo and chunk size to line up

setQuantizedSampleRate

```
public final void setQuantizedSampleRate(float beatsPerMinute)
```

Sets the sample rate that will result in producing chunks captured at a length of time equivalent to one beat of the given tempo. This calculation is also related to the chunk size. A decrease chunk size will increase the resulting sample rate.

Parameters:

beatsPerMinute - the tempo used in BeatsPerMinute. Note that if a division smaller than one beat is preferred then multiply the tempo by the subdivision.

setSampleRate

```
public void setSampleRate(float sampleRate)
```

Set the sample rate to be used for capturing. Note: This may produce unusual affects if changed while the capture is currently processing chunks.

Parameters:

sampleRate - measured in Hertz.

getChunkSize

```
public int getChunkSize()
```

Gets Chunk Size

Returns:

the size in bytes that chunks will be captured at

getAudioFormat

```
public AudioFormat getAudioFormat()
```

Get the audio formate to be captured

Returns:

the audio format to be captured

setAudioFormat

```
public void setAudioFormat(AudioFormat audioFormat)
```

Set the audio format for capturing.

byteArrToDouble

```
public static double byteArrToDouble(byte[] arr,  
                                     int start)
```

Parameters:

arr - array of bytes to read from

start - the index that starts the double

Returns:

a double consisting of the next 8 bytes in the array

byteArrToInt

```
public static int byteArrToInt(byte[] arr,  
                               int start)
```

Found at <http://www.captain.at/howto-java-convert-binary-data.php>

Parameters:

arr - array of bytes to read from

start - the index that starts the int

Returns:

a int consisting of the next 2 bytes in the array

Class Chunk

sp

```
java.lang.Object
└─ sp.Chunk
```

All Implemented Interfaces:

Frequency

```
public class Chunk
extends Object
implements Frequency
```

Embodies on chunk of windowed audio data. Performs FFT on array of PCM data and computes information about chunk.

Author:

Sean T. Hayes

See Also:

Capture

Field Summary		Page
static int	<u>DEFAULT_CHUNK_SIZE</u> Default number (1024) of doubles per chunk in the window.	41
static int	<u>DEFAULT_NUM_OF_FREQUENCIES</u> Default number (6) of saved frequencies	41
boolean	<u>VIRBOSE</u> Used to for debugging.	41

Constructor Summary		Page
<u>Chunk</u> ()	Default Constructor initializes the sample rate to 8000Hz	41
<u>Chunk</u> (double sampleRate, double[] iAudio, int chunkSize)	Constructor	42
<u>Chunk</u> (double sampleRate, double[] iAudio, int chunkSize, boolean isLast)	Constructor	42
<u>Chunk</u> (double sampleRate, int chunkSize)	Sets the number of saved frequencies to the default	42

Method Summary		Page
static int	<u>gcd</u> (int n, int d) Euclid's greatest common divisor algorithm.	45
int	<u>getCents</u> () The number of cents sharp or flat (negative for flat and positive for sharp).	44
int	<u>getChunkSize</u> () Get the number of elements used to create this chunk	44
double[]	<u>getFrequencies</u> () Get maximum frequencies for this chunk	43

double	<u>getFundamental()</u> Analyzes the maximum frequencies to extrapolate the fundamental.	44
boolean	<u>getIsLast()</u> Indicates that this is the last chunk to analyze.	44
int	<u>getMidiNumber()</u> Get the MIDI number closest to this frequency.	43
double	<u>getSampleRate()</u> Get the sample rate used to capture this chunk	43
static boolean	<u>isPowerOfTwo(int num)</u> Checks to see if a number is a power of 2	45
void	<u>setChunk(double[] iAudio)</u> Recalculate the frequencies using a different pice of data	42
void	<u>setChunkSize(int chunkSize)</u> This must be set before captured audio is sent to the chunk	45
void	<u>setIsLast(boolean isLast)</u> Sets a marker that indicates to the analyzer that this is the last chunk.	44
void	<u>setNumOfFrequencies(int num)</u> Change the number of maximum frequencies to be stored	43
void	<u>setSampleRate(double sampleRate)</u> Should be called before getChunk()	43

Field Detail

VIRBOSE

```
public final boolean VIRBOSE
```

Used to for debugging. Outputs more information to standard output device

DEFAULT_CHUNK_SIZE

```
public static final int DEFAULT_CHUNK_SIZE
```

Default number (1024) of doubles per chunk in the window. FFT output array will be half of the chunk size.

DEFAULT_NUM_OF_FREQUENCIES

```
public static final int DEFAULT_NUM_OF_FREQUENCIES
```

Default number (6) of saved frequencies

Constructor Detail

Chunk

```
public Chunk()
```

Default Constructor initializes the sample rate to 8000Hz

Chunk

```
public Chunk(double sampleRate,  
             int chunkSize)
```

Sets the number of saved frequencies to the default

Parameters:

sampleRate - capture sample rate (in Hz)

Chunk

```
public Chunk(double sampleRate,  
             double[] iAudio,  
             int chunkSize)
```

Constructor

Parameters:

sampleRate - sample rate of iAudio data

iAudio - an array of PCM audio data (values between -1 and 1)

chunkSize - the size of the chunk in bytes

Chunk

```
public Chunk(double sampleRate,  
             double[] iAudio,  
             int chunkSize,  
             boolean isLast)
```

Constructor

Parameters:

sampleRate - sample rate of iAudio data

iAudio - an array of PCM audio data (values between -1 and 1)

chunkSize - how many bytes of data should be read from the audio buffer for each chunk

isLast - Indicates that this is the last chunk to analyze. This is not used in any internal calculations but acts as a marker for anything processing a series of chunks

Method Detail

setChunk

```
public void setChunk(double[] iAudio)
```

Recalculate the frequencies using a different pice of data

Parameters:

iAudio - should match chunk size and be a power of 2

setNumOfFrequencies

```
public void setNumOfFrequencies(int num)
```

Change the number of maximum frequencies to be stored

Parameters:

num - should be great than 0 and less than the number of array elements sent the Chunk.

getFrequencies

```
public double[] getFrequencies()
```

Get maximum frequencies for this chunk

Returns:

an array of the frequencies with the greatest amplitude

getMidiNumber

```
public int getMidiNumber()
```

Description copied from interface: Frequency
Get the MIDI number closest to this frequency.

Specified by:

getMidiNumber in interface Frequency

Returns:

the MIDI number of the fundamental frequency represented by chunk.

setSampleRate

```
public void setSampleRate(double sampleRate)
```

Should be called before getChunck()

Parameters:

sampleRate - the sample rate to record (in Hz).

getSampleRate

```
public double getSampleRate()
```

Get the sample rate used to capture this chunk

Returns:

the sample rate in Hz.

getFundamental

```
public double getFundamental()
```

Analyzes the maximum frequencies to extrapolate the fundamental.

Specified by:

getFundamental in interface Frequency

Returns:

the fundamental frequency (0 if no sufficient frequency is given)

getCents

```
public int getCents()
```

Description copied from interface: Frequency

The number of cents sharp or flat (negative for flat and positive for sharp).

Specified by:

getCents in interface Frequency

Returns:

the number of cents flat or sharp the frequency is for its closest neighbor. As a result this will be between -50 and 50 inclusive.

setIsLast

```
public void setIsLast(boolean isLast)
```

Sets a marker that indicates to the analyzer that this is the last chunk. After processing this chunk the analyzer thread will end.

Parameters:

isLast - true if this is the last chunk to analyze

getIsLast

```
public boolean getIsLast()
```

Indicates that this is the last chunk to analyze.

Returns:

isLast true if this is the last chunk to analyze

getChunkSize

```
public int getChunkSize()
```

Get the number of elements used to create this chunk

Returns:
the number of samples this chunk

setChunkSize

```
public void setChunkSize(int chunkSize)
```

This must be set before captured audio is sent to the chunk

Parameters:
chunkSize - the size in bytes

gcd

```
public static final int gcd(int n,  
                             int d)
```

Euclid's greatest common divisor algorithm. Find the largest number that evenly divides into both n and d.

Parameters:
n - must be ≥ 0
d - must be ≥ 0

Returns:
the greatest common factor

isPowerOfTwo

```
public static final boolean isPowerOfTwo(int num)
```

Checks to see if a number is a power of 2

Returns:
true if the number is a multiple of 2

Class CustomFileFilter

sp

```
java.lang.Object
├─ javax.swing.filechooser.FileFilter
│   └─ sp.CustomFileFilter
```

```
public class CustomFileFilter
    extends FileFilter
```

This class allows for the creation of a file filter that can easily be edited during runtime.

Author:

Sean Timothy Hayes

Constructor Summary	Page
CustomFileFilter()	46
CustomFileFilter(String description)	46

Method Summary	Page
boolean accept (File f)	46
void addExtension (String extention) Add an extention to the list of allowed.	47
String getDescription ()	47
static String getExtension (File f)	47
void setDescription (String description)	47

Constructor Detail

CustomFileFilter

```
public CustomFileFilter()
```

CustomFileFilter

```
public CustomFileFilter(String description)
```

Method Detail

accept

```
public boolean accept(File f)
```

Overrides:

`accept` in class `FileFilter`

getDescription

```
public String getDescription()
```

Overrides:

```
getDescription in class FileFilter
```

setDescription

```
public void setDescription(String description)
```

addExtension

```
public void addExtension(String extention)
```

Add an extention to the list of allowed.

getExtension

```
public static String getExtension(File f)
```

Interface Frequency

sp

All Known Implementing Classes:

[Chunk](#)

public interface **Frequency**

Contains methods for extrapolating information based on a single frequency

Author:

Sean T. Hayes

See Also:

[Chunk](#)

Method Summary		Page
int	<u>getCents()</u> The number of cents sharp or flat (negative for flat and positive for sharp).	48
double	<u>getFundamental()</u> Get the Frequency	48
int	<u>getMidiNumber()</u> Get the MIDI number closest to this frequency.	48

Method Detail

getFundamental

double **getFundamental()**

Get the Frequency

Returns:

the frequency of the current object

getCents

int **getCents()**

The number of cents sharp or flat (negative for flat and positive for sharp).

Returns:

the number of cents flat or sharp the frequency is for its closest neighbor. As a result this will be between -50 and 50 inclusive.

getMidiNumber

int **getMidiNumber()**

Get the MIDI number closest to this frequency.

Returns:

MIDI number

Class Metronome

sp

```
java.lang.Object
└─ sp.Metronome
```

```
public class Metronome
extends Object
```

This class is the heart of rhythmic measurement and is used for any action that is required to happen on the beat.

Author:

Sean Timothy Hayes

Field Summary		Page
protected EventListenerList	<u>listenerList</u> Create the listener list	51

Constructor Summary		Page
<u>Metronome</u> ()	Default constructor, initializes the tempo to 120 beats per minute	51
<u>Metronome</u> (double tempo)		51

Method Summary		Page
void	<u>addPulseListener</u> (PulseListener listener) This methods allows classes to register for Pulse events	52
boolean	<u>getIsOn</u> () Is the metronome currently on	51
double	<u>getTempo</u> () Get the tempo	51
short	<u>getVolume</u> () Get the volume	52
void	<u>removeMyEventListener</u> (PulseListener listener) This methods allows classes to unregister for Pulse events	53
void	<u>setOn</u> (boolean on) Another was to control the metronome	52
void	<u>setTempo</u> (double tempo) Set the tempo	51
void	<u>setVolume</u> (short volume) Set the volume that the pulse should sound at	52
void	<u>start</u> () If the metronome is not yet running start it.	52
void	<u>stop</u> () Stop this metronome	52

Field Detail

listenerList

`protected EventListenerList listenerList`

Create the listener list

Constructor Detail

Metronome

`public Metronome()`

Default constructor, initializes the tempo to 120 beats per minute

Metronome

`public Metronome(double tempo)`

Parameters:

`tempo` - measured in Beats Per Minute

Method Detail

getTempo

`public double getTempo()`

Get the tempo

Returns:

the tempo in beats per minute

setTempo

`public void setTempo(double tempo)`

Set the tempo

Parameters:

`tempo` - measured in beats per minute. This must be greater than zero.

getIsOn

`public boolean getIsOn()`

Is the metronome currently on

Returns:

`true` if the metronome is current running.

start

`public synchronized void start()`

If the metronome is not yet running start it.

stop

`public synchronized void stop()`

Stop this metronome

setOn

`public void setOn(boolean on)`

Another way to control the metronome

Parameters:

`on` - if `true` the metronome will start. If `false` the metronome will stop.

See Also:

[`start\(\)`](#), [`stop\(\)`](#)

setVolume

`public void setVolume(short volume)`

Set the volume that the pulse should sound at

Parameters:

`volume` - value between 0 and 100 inclusive

getVolume

`public short getVolume()`

Get the volume

Returns:

a value between 0 and 100 inclusive

addPulseListener

`public void addPulseListener(PulseListener listener)`

This methods allows classes to register for Pulse events

Parameters:

listener - the `PulseListener` to be added

removeMyEventListener

```
public void removeMyEventListener(PulseListener listener)
```

This methods allows classes to unregister for Pulse events

Parameters:

listener - the `PulseListener` to be removed

Class **PlayPulseThread**

sp

```
java.lang.Object
├ java.lang.Thread
│   └ sp.PlayPulseThread
```

All Implemented Interfaces:
Runnable

```
public class PlayPulseThread
extends Thread
```

This class is used by the *Metronome* class to produce and even pulse in the specified tempo.

Author:

Sean Timothy Hayes

See Also:

Metronome

Field Summary		Page
protected EventListenerList	<u>listenerList</u> Create the listener list	54

Constructor Summary		Page
<u>PlayPulseThread</u> (double tempo, EventListenerList listenerList, short volume)		55

Method Summary		Page
void	<u>addPulseListener</u> (PulseListener listener) This methods allows classes to register for Pulse events	55
double	<u>getTempo</u> ()	55
short	<u>getVolume</u> ()	55
void	<u>removeMyEventListener</u> (PulseListener listener) This methods allows classes to unregister for Pulse events	56
void	<u>run</u> ()	55
void	<u>setTempo</u> (double tempo)	55
void	<u>setVolume</u> (short volume)	55
void	<u>stopPlaying</u> ()	55

Field Detail

listenerList

```
protected EventListenerList listenerList
```

Create the listener list

Constructor Detail

PlayPulseThread

```
public PlayPulseThread(double tempo,
                        EventListenerList listenerList,
                        short volume)
```

Parameters:

tempo - the pulse in beats per minute
volume - a value between 0 and 100 inclusive where 100 is full volume

Method Detail

run

```
public void run()
```

Specified by:

run in interface Runnable

Overrides:

run in class Thread

stopPlaying

```
public void stopPlaying()
```

setTempo

```
public void setTempo(double tempo)
```

getTempo

```
public double getTempo()
```

setVolume

```
public void setVolume(short volume)
```

getVolume

```
public short getVolume()
```

addPulseListener

```
public void addPulseListener(PulseListener listener)
```

This methods allows classes to register for Pulse events

Parameters:

listener - the `PulseListener` to be added

removeMyEventListener

```
public void removeMyEventListener(PulseListener listener)
```

This methods allows classes to unregister for Pulse events

Parameters:

listener - the `PulseListener` to be removed

Class **PulseEvent**

sp

```
java.lang.Object
├─ java.util.EventObject
│   └─ sp.PulseEvent
```

All Implemented Interfaces:
Serializable

```
public class PulseEvent
    extends EventObject
```

This event occurs at every pulse of the metronome. Listeners can be added to class `Metronome` to monitor this event.

Author:
Sean Hayes

See Also:
[Metronome](#), [PulseEvent](#), [PlayPulseThread](#)

Constructor Summary	Page
PulseEvent (Object source)	57

Constructor Detail

PulseEvent

```
public PulseEvent(Object source)
```

Interface **PulseListener**

sp

All Superinterfaces:
EventListener

```
public interface PulseListener
    extends EventListener
```

The listener interface for receiving pulse events from the metronome. The class that is interested in processing an pulse event implements this interface, and the object created with that class is registered with an instance of the metronome class, using the metronome's `addPulseListener` method. When the action event occurs, that object's `onPulse` method is invoked.

Author:
Sean Timothy Hayes

Method Summary		Page
void	<code>onPulse(PulseEvent event)</code>	58

Method Detail

onPulse

```
void onPulse(PulseEvent event)
```

Package imp

Interface Summary		Page
<u>Constants</u>	A class to hold all Impro-Visor constants.	60

Enum Summary		Page
<u>Constants.Accidental</u>	a flag so a Note knows whether it should be sharp, flat, or natural	153
<u>Constants.ExtractMode</u>	Modes of extracting stuff for transfer	155
<u>Constants.StaveType</u>	enum indicating which stave the current window should display	157

Interface Constants

imp

public interface Constants

A class to hold all Impro-Visor constants. To use this interface, import `imp.Constants` and make sure your class implements `Constants`. Then you can just use `BEAT`, `REST`, etc. in your code.

Nested Class Summary		Page
static enum	<u>Constants.Accidental</u> a flag so a Note knows whether it should be sharp, flat, or natural	153
static enum	<u>Constants.ExtractMode</u> Modes of extracting stuff for transfer	155
static enum	<u>Constants.StaveType</u> enum indicating which stave the current window should display	157

Field Summary		Page
int	<u>A0</u> maps absolute notes to MIDI numbers	114
int	<u>a0</u> maps absolute notes to MIDI numbers	144
int	<u>A1</u> maps absolute notes to MIDI numbers	112
int	<u>a1</u> maps absolute notes to MIDI numbers	141
int	<u>A2</u> maps absolute notes to MIDI numbers	109
int	<u>a2</u> maps absolute notes to MIDI numbers	138
int	<u>A3</u> maps absolute notes to MIDI numbers	106
int	<u>a3</u> maps absolute notes to MIDI numbers	135
int	<u>A4</u> maps absolute notes to MIDI numbers	103
int	<u>a4</u> maps absolute notes to MIDI numbers	133
int	<u>A5</u> maps absolute notes to MIDI numbers	101
int	<u>a5</u> maps absolute notes to MIDI numbers	130
int	<u>A6</u> maps absolute notes to MIDI numbers	98
int	<u>a6</u> maps absolute notes to MIDI numbers	127

int	<u>A7</u>	maps absolute notes to MIDI numbers	95
int	<u>a7</u>	maps absolute notes to MIDI numbers	124
int	<u>A8</u>	maps absolute notes to MIDI numbers	93
int	<u>a8</u>	maps absolute notes to MIDI numbers	122
int	<u>ABMAJOR</u>		150
int	<u>AF0</u>	maps absolute notes to MIDI numbers	115
int	<u>af0</u>	maps absolute notes to MIDI numbers	144
int	<u>AF1</u>	maps absolute notes to MIDI numbers	112
int	<u>af1</u>	maps absolute notes to MIDI numbers	141
int	<u>AF2</u>	maps absolute notes to MIDI numbers	109
int	<u>af2</u>	maps absolute notes to MIDI numbers	138
int	<u>AF3</u>	maps absolute notes to MIDI numbers	106
int	<u>af3</u>	maps absolute notes to MIDI numbers	135
int	<u>AF4</u>	maps absolute notes to MIDI numbers	104
int	<u>af4</u>	maps absolute notes to MIDI numbers	133
int	<u>AF5</u>	maps absolute notes to MIDI numbers	101
int	<u>af5</u>	maps absolute notes to MIDI numbers	130
int	<u>AF6</u>	maps absolute notes to MIDI numbers	98
int	<u>af6</u>	maps absolute notes to MIDI numbers	127
int	<u>AF7</u>	maps absolute notes to MIDI numbers	95
int	<u>af7</u>	maps absolute notes to MIDI numbers	124
int	<u>AF8</u>	maps absolute notes to MIDI numbers	93
int	<u>af8</u>	maps absolute notes to MIDI numbers	122
int	<u>AFN1</u>	maps absolute notes to MIDI numbers	117

int	<u>afn1</u> maps absolute notes to MIDI numbers	146
int	<u>ALL</u> Which portions of the leadsheet to export	87
int	<u>AMAJOR</u>	151
int	<u>AN1</u> maps absolute notes to MIDI numbers	117
int	<u>an1</u> maps absolute notes to MIDI numbers	146
int	<u>APPROACH TONE</u>	86
int	<u>AS0</u> maps absolute notes to MIDI numbers	114
int	<u>as0</u> maps absolute notes to MIDI numbers	143
int	<u>AS1</u> maps absolute notes to MIDI numbers	112
int	<u>as1</u> maps absolute notes to MIDI numbers	141
int	<u>AS2</u> maps absolute notes to MIDI numbers	109
int	<u>as2</u> maps absolute notes to MIDI numbers	138
int	<u>AS3</u> maps absolute notes to MIDI numbers	106
int	<u>as3</u> maps absolute notes to MIDI numbers	135
int	<u>AS4</u> maps absolute notes to MIDI numbers	103
int	<u>as4</u> maps absolute notes to MIDI numbers	132
int	<u>AS5</u> maps absolute notes to MIDI numbers	101
int	<u>as5</u> maps absolute notes to MIDI numbers	130
int	<u>AS6</u> maps absolute notes to MIDI numbers	98
int	<u>as6</u> maps absolute notes to MIDI numbers	127
int	<u>AS7</u> maps absolute notes to MIDI numbers	95
int	<u>as7</u> maps absolute notes to MIDI numbers	124
int	<u>AS8</u> maps absolute notes to MIDI numbers	92
int	<u>as8</u> maps absolute notes to MIDI numbers	121
int	<u>ASHARP</u>	86

int	<u>ASN1</u> maps absolute notes to MIDI numbers	117
int	<u>asn1</u> maps absolute notes to MIDI numbers	146
int	<u>B0</u> maps absolute notes to MIDI numbers	114
int	<u>b0</u> maps absolute notes to MIDI numbers	143
int	<u>B1</u> maps absolute notes to MIDI numbers	111
int	<u>b1</u> maps absolute notes to MIDI numbers	140
int	<u>B2</u> maps absolute notes to MIDI numbers	109
int	<u>b2</u> maps absolute notes to MIDI numbers	138
int	<u>B3</u> maps absolute notes to MIDI numbers	106
int	<u>b3</u> maps absolute notes to MIDI numbers	135
int	<u>B4</u> maps absolute notes to MIDI numbers	103
int	<u>b4</u> maps absolute notes to MIDI numbers	132
int	<u>B5</u> maps absolute notes to MIDI numbers	100
int	<u>b5</u> maps absolute notes to MIDI numbers	129
int	<u>B6</u> maps absolute notes to MIDI numbers	98
int	<u>b6</u> maps absolute notes to MIDI numbers	127
int	<u>B7</u> maps absolute notes to MIDI numbers	95
int	<u>b7</u> maps absolute notes to MIDI numbers	124
int	<u>B8</u> maps absolute notes to MIDI numbers	92
int	<u>b8</u> maps absolute notes to MIDI numbers	121
char	<u>BACKSLASH</u>	90
char	<u>BAR</u>	90
String	<u>BARSTRING</u> The bar (measure) symbol	88
int	<u>BBMAJOR</u>	150
int	<u>BEAT</u> defines how many slots a BEAT takes up	83

int	<u>BF0</u>	maps absolute notes to MIDI numbers	114
int	<u>bf0</u>	maps absolute notes to MIDI numbers	143
int	<u>BF1</u>	maps absolute notes to MIDI numbers	111
int	<u>bf1</u>	maps absolute notes to MIDI numbers	141
int	<u>BF2</u>	maps absolute notes to MIDI numbers	109
int	<u>bf2</u>	maps absolute notes to MIDI numbers	138
int	<u>BF3</u>	maps absolute notes to MIDI numbers	106
int	<u>bf3</u>	maps absolute notes to MIDI numbers	135
int	<u>BF4</u>	maps absolute notes to MIDI numbers	103
int	<u>bf4</u>	maps absolute notes to MIDI numbers	132
int	<u>BF5</u>	maps absolute notes to MIDI numbers	100
int	<u>bf5</u>	maps absolute notes to MIDI numbers	130
int	<u>BF6</u>	maps absolute notes to MIDI numbers	98
int	<u>bf6</u>	maps absolute notes to MIDI numbers	127
int	<u>BF7</u>	maps absolute notes to MIDI numbers	95
int	<u>bf7</u>	maps absolute notes to MIDI numbers	124
int	<u>BF8</u>	maps absolute notes to MIDI numbers	92
int	<u>bf8</u>	maps absolute notes to MIDI numbers	121
int	<u>BFN1</u>	maps absolute notes to MIDI numbers	117
int	<u>bfN1</u>	maps absolute notes to MIDI numbers	146
int []	<u>blackNotes</u>	Positions of black notes in the chromatic scale (starting at 0)	87
int	<u>BMAJOR</u>		151
int	<u>BN1</u>	maps absolute notes to MIDI numbers	117
int	<u>bn1</u>	maps absolute notes to MIDI numbers	146

int	<u>BS0</u>	maps absolute notes to MIDI numbers	114
int	<u>bs0</u>	maps absolute notes to MIDI numbers	143
int	<u>BS1</u>	maps absolute notes to MIDI numbers	111
int	<u>bs1</u>	maps absolute notes to MIDI numbers	140
int	<u>BS2</u>	maps absolute notes to MIDI numbers	108
int	<u>bs2</u>	maps absolute notes to MIDI numbers	138
int	<u>BS3</u>	maps absolute notes to MIDI numbers	106
int	<u>bs3</u>	maps absolute notes to MIDI numbers	135
int	<u>BS4</u>	maps absolute notes to MIDI numbers	103
int	<u>bs4</u>	maps absolute notes to MIDI numbers	132
int	<u>BS5</u>	maps absolute notes to MIDI numbers	100
int	<u>bs5</u>	maps absolute notes to MIDI numbers	129
int	<u>BS6</u>	maps absolute notes to MIDI numbers	97
int	<u>bs6</u>	maps absolute notes to MIDI numbers	127
int	<u>BS7</u>	maps absolute notes to MIDI numbers	95
int	<u>bs7</u>	maps absolute notes to MIDI numbers	124
int	<u>BS8</u>	maps absolute notes to MIDI numbers	92
int	<u>bs8</u>	maps absolute notes to MIDI numbers	121
int	<u>BSN1</u>	maps absolute notes to MIDI numbers	117
int	<u>bsn1</u>	maps absolute notes to MIDI numbers	146
int	<u>C0</u>	maps absolute notes to MIDI numbers	116
int	<u>c0</u>	maps absolute notes to MIDI numbers	145
int	<u>C1</u>	maps absolute notes to MIDI numbers	114
int	<u>c1</u>	maps absolute notes to MIDI numbers	143

int	<u>C2</u>	maps absolute notes to MIDI numbers	111
int	<u>c2</u>	maps absolute notes to MIDI numbers	140
int	<u>C3</u>	maps absolute notes to MIDI numbers	108
int	<u>c3</u>	maps absolute notes to MIDI numbers	137
int	<u>C4</u>	maps absolute notes to MIDI numbers	105
int	<u>c4</u>	maps absolute notes to MIDI numbers	135
int	<u>C5</u>	maps absolute notes to MIDI numbers	103
int	<u>c5</u>	maps absolute notes to MIDI numbers	132
int	<u>C6</u>	maps absolute notes to MIDI numbers	100
int	<u>c6</u>	maps absolute notes to MIDI numbers	129
int	<u>C7</u>	maps absolute notes to MIDI numbers	97
int	<u>c7</u>	maps absolute notes to MIDI numbers	126
int	<u>C8</u>	maps absolute notes to MIDI numbers	94
int	<u>c8</u>	maps absolute notes to MIDI numbers	124
int	<u>C9</u>	maps absolute notes to MIDI numbers	92
int	<u>c9</u>	maps absolute notes to MIDI numbers	121
int	<u>CBMAJOR</u>		150
int	<u>CF0</u>	maps absolute notes to MIDI numbers	117
int	<u>cf0</u>	maps absolute notes to MIDI numbers	146
int	<u>CF1</u>	maps absolute notes to MIDI numbers	114
int	<u>cf1</u>	maps absolute notes to MIDI numbers	143
int	<u>CF2</u>	maps absolute notes to MIDI numbers	111
int	<u>cf2</u>	maps absolute notes to MIDI numbers	140
int	<u>CF3</u>	maps absolute notes to MIDI numbers	108

int	<u>cf3</u> maps absolute notes to MIDI numbers	137
int	<u>CF4</u> maps absolute notes to MIDI numbers	106
int	<u>cf4</u> maps absolute notes to MIDI numbers	135
int	<u>CF5</u> maps absolute notes to MIDI numbers	103
int	<u>cf5</u> maps absolute notes to MIDI numbers	132
int	<u>CF6</u> maps absolute notes to MIDI numbers	100
int	<u>cf6</u> maps absolute notes to MIDI numbers	129
int	<u>CF7</u> maps absolute notes to MIDI numbers	97
int	<u>cf7</u> maps absolute notes to MIDI numbers	126
int	<u>CF8</u> maps absolute notes to MIDI numbers	95
int	<u>cf8</u> maps absolute notes to MIDI numbers	124
int	<u>CF9</u> maps absolute notes to MIDI numbers	92
int	<u>cf9</u> maps absolute notes to MIDI numbers	121
int	<u>CHORD TONE</u> Types of notes	86
int	<u>CHORDS ONLY</u>	87
int	<u>CMAJOR</u>	150
int	<u>CMIDI</u> The midi number for middle C	89
int	<u>CN1</u> maps absolute notes to MIDI numbers	119
int	<u>cn1</u> maps absolute notes to MIDI numbers	148
int	<u>COLOR TONE</u>	86
char	<u>COMMA</u>	90
String	<u>COMMASTRING</u> Comma is also usable as a measure symbol	88
String	<u>CROOT</u> The common root for chord table lookup.	89
int	<u>CS0</u> maps absolute notes to MIDI numbers	116
int	<u>cs0</u> maps absolute notes to MIDI numbers	145
int	<u>CS1</u> maps absolute notes to MIDI numbers	114

int	<u>cs1</u>	maps absolute notes to MIDI numbers	143
int	<u>CS2</u>	maps absolute notes to MIDI numbers	111
int	<u>cs2</u>	maps absolute notes to MIDI numbers	140
int	<u>CS3</u>	maps absolute notes to MIDI numbers	108
int	<u>cs3</u>	maps absolute notes to MIDI numbers	137
int	<u>CS4</u>	maps absolute notes to MIDI numbers	105
int	<u>cs4</u>	maps absolute notes to MIDI numbers	134
int	<u>CS5</u>	maps absolute notes to MIDI numbers	103
int	<u>cs5</u>	maps absolute notes to MIDI numbers	132
int	<u>CS6</u>	maps absolute notes to MIDI numbers	100
int	<u>cs6</u>	maps absolute notes to MIDI numbers	129
int	<u>CS7</u>	maps absolute notes to MIDI numbers	97
int	<u>cs7</u>	maps absolute notes to MIDI numbers	126
int	<u>CS8</u>	maps absolute notes to MIDI numbers	94
int	<u>cs8</u>	maps absolute notes to MIDI numbers	123
int	<u>CS9</u>	maps absolute notes to MIDI numbers	92
int	<u>cs9</u>	maps absolute notes to MIDI numbers	121
int	<u>CSHARP</u>	Enharmonics	86
int	<u>CSMAJOR</u>		151
int	<u>CSN1</u>	maps absolute notes to MIDI numbers	119
int	<u>csn1</u>	maps absolute notes to MIDI numbers	148
int	<u>D0</u>	maps absolute notes to MIDI numbers	116
int	<u>d0</u>	maps absolute notes to MIDI numbers	145
int	<u>D1</u>	maps absolute notes to MIDI numbers	113

int	<u>d1</u> maps absolute notes to MIDI numbers	142
int	<u>D2</u> maps absolute notes to MIDI numbers	111
int	<u>d2</u> maps absolute notes to MIDI numbers	140
int	<u>D3</u> maps absolute notes to MIDI numbers	108
int	<u>d3</u> maps absolute notes to MIDI numbers	137
int	<u>D4</u> maps absolute notes to MIDI numbers	105
int	<u>d4</u> maps absolute notes to MIDI numbers	134
int	<u>D5</u> maps absolute notes to MIDI numbers	102
int	<u>d5</u> maps absolute notes to MIDI numbers	131
int	<u>D6</u> maps absolute notes to MIDI numbers	100
int	<u>d6</u> maps absolute notes to MIDI numbers	129
int	<u>D7</u> maps absolute notes to MIDI numbers	97
int	<u>d7</u> maps absolute notes to MIDI numbers	126
int	<u>D8</u> maps absolute notes to MIDI numbers	94
int	<u>d8</u> maps absolute notes to MIDI numbers	123
int	<u>D9</u> maps absolute notes to MIDI numbers	91
int	<u>d9</u> maps absolute notes to MIDI numbers	120
int	<u>DBMAJOR</u>	150
int	<u>DEFAULT BEAT DENOMINATOR</u>	82
int	<u>DEFAULT BEATS PER BAR</u>	82
int	<u>DEFAULT DURATION</u>	85
String	<u>DEFAULT DURATION MSG</u> The default duration of a note when none is specified.	89
int[]	<u>DEFAULT METRE</u>	86
double	<u>DEFAULT SWING</u>	85
int	<u>DF0</u> maps absolute notes to MIDI numbers	116
int	<u>df0</u> maps absolute notes to MIDI numbers	145

int	<u>DF1</u>	maps absolute notes to MIDI numbers	113
int	<u>df1</u>	maps absolute notes to MIDI numbers	142
int	<u>DF2</u>	maps absolute notes to MIDI numbers	111
int	<u>df2</u>	maps absolute notes to MIDI numbers	140
int	<u>DF3</u>	maps absolute notes to MIDI numbers	108
int	<u>df3</u>	maps absolute notes to MIDI numbers	137
int	<u>DF4</u>	maps absolute notes to MIDI numbers	105
int	<u>df4</u>	maps absolute notes to MIDI numbers	134
int	<u>DF5</u>	maps absolute notes to MIDI numbers	102
int	<u>df5</u>	maps absolute notes to MIDI numbers	132
int	<u>DF6</u>	maps absolute notes to MIDI numbers	100
int	<u>df6</u>	maps absolute notes to MIDI numbers	129
int	<u>DF7</u>	maps absolute notes to MIDI numbers	97
int	<u>df7</u>	maps absolute notes to MIDI numbers	126
int	<u>DF8</u>	maps absolute notes to MIDI numbers	94
int	<u>df8</u>	maps absolute notes to MIDI numbers	123
int	<u>DF9</u>	maps absolute notes to MIDI numbers	91
int	<u>df9</u>	maps absolute notes to MIDI numbers	121
int	<u>DFN1</u>	maps absolute notes to MIDI numbers	119
int	<u>dfn1</u>	maps absolute notes to MIDI numbers	148
int	<u>DMAJOR</u>		151
int	<u>DN1</u>	maps absolute notes to MIDI numbers	119
int	<u>dn1</u>	maps absolute notes to MIDI numbers	148
char	<u>DOT</u>		90
int	<u>DOTTED EIGHTH</u>		84
int	<u>DOTTED HALF</u>		84

int	<u>DOTTED_QUARTER</u>	84
int	<u>DOTTED_SIXTEENTH</u>	84
int	<u>DS0</u> maps absolute notes to MIDI numbers	116
int	<u>ds0</u> maps absolute notes to MIDI numbers	145
int	<u>DS1</u> maps absolute notes to MIDI numbers	113
int	<u>ds1</u> maps absolute notes to MIDI numbers	142
int	<u>DS2</u> maps absolute notes to MIDI numbers	110
int	<u>ds2</u> maps absolute notes to MIDI numbers	139
int	<u>DS3</u> maps absolute notes to MIDI numbers	108
int	<u>ds3</u> maps absolute notes to MIDI numbers	137
int	<u>DS4</u> maps absolute notes to MIDI numbers	105
int	<u>ds4</u> maps absolute notes to MIDI numbers	134
int	<u>DS5</u> maps absolute notes to MIDI numbers	102
int	<u>ds5</u> maps absolute notes to MIDI numbers	131
int	<u>DS6</u> maps absolute notes to MIDI numbers	99
int	<u>ds6</u> maps absolute notes to MIDI numbers	129
int	<u>DS7</u> maps absolute notes to MIDI numbers	97
int	<u>ds7</u> maps absolute notes to MIDI numbers	126
int	<u>DS8</u> maps absolute notes to MIDI numbers	94
int	<u>ds8</u> maps absolute notes to MIDI numbers	123
int	<u>DS9</u> maps absolute notes to MIDI numbers	91
int	<u>ds9</u> maps absolute notes to MIDI numbers	120
int	<u>DSHARP</u>	86
int	<u>DSN1</u> maps absolute notes to MIDI numbers	119
int	<u>dsn1</u> maps absolute notes to MIDI numbers	148

int	<u>E0</u>	maps absolute notes to MIDI numbers	116
int	<u>e0</u>	maps absolute notes to MIDI numbers	145
int	<u>E1</u>	maps absolute notes to MIDI numbers	113
int	<u>e1</u>	maps absolute notes to MIDI numbers	142
int	<u>E2</u>	maps absolute notes to MIDI numbers	110
int	<u>e2</u>	maps absolute notes to MIDI numbers	139
int	<u>E3</u>	maps absolute notes to MIDI numbers	107
int	<u>e3</u>	maps absolute notes to MIDI numbers	136
int	<u>E4</u>	maps absolute notes to MIDI numbers	105
int	<u>e4</u>	maps absolute notes to MIDI numbers	134
int	<u>E5</u>	maps absolute notes to MIDI numbers	102
int	<u>e5</u>	maps absolute notes to MIDI numbers	131
int	<u>E6</u>	maps absolute notes to MIDI numbers	99
int	<u>e6</u>	maps absolute notes to MIDI numbers	128
int	<u>E7</u>	maps absolute notes to MIDI numbers	96
int	<u>e7</u>	maps absolute notes to MIDI numbers	126
int	<u>E8</u>	maps absolute notes to MIDI numbers	94
int	<u>e8</u>	maps absolute notes to MIDI numbers	123
int	<u>E9</u>	maps absolute notes to MIDI numbers	91
int	<u>e9</u>	maps absolute notes to MIDI numbers	120
int	<u>EBMAJOR</u>		150
int	<u>EF0</u>	maps absolute notes to MIDI numbers	116
int	<u>eF0</u>	maps absolute notes to MIDI numbers	145
int	<u>EF1</u>	maps absolute notes to MIDI numbers	113

int	<u>ef1</u> maps absolute notes to MIDI numbers	142
int	<u>EF2</u> maps absolute notes to MIDI numbers	110
int	<u>ef2</u> maps absolute notes to MIDI numbers	139
int	<u>EF3</u> maps absolute notes to MIDI numbers	108
int	<u>ef3</u> maps absolute notes to MIDI numbers	137
int	<u>EF4</u> maps absolute notes to MIDI numbers	105
int	<u>ef4</u> maps absolute notes to MIDI numbers	134
int	<u>EF5</u> maps absolute notes to MIDI numbers	102
int	<u>ef5</u> maps absolute notes to MIDI numbers	131
int	<u>EF6</u> maps absolute notes to MIDI numbers	99
int	<u>ef6</u> maps absolute notes to MIDI numbers	128
int	<u>EF7</u> maps absolute notes to MIDI numbers	97
int	<u>ef7</u> maps absolute notes to MIDI numbers	126
int	<u>EF8</u> maps absolute notes to MIDI numbers	94
int	<u>ef8</u> maps absolute notes to MIDI numbers	123
int	<u>EF9</u> maps absolute notes to MIDI numbers	91
int	<u>ef9</u> maps absolute notes to MIDI numbers	120
int	<u>EFN1</u> maps absolute notes to MIDI numbers	118
int	<u>efn1</u> maps absolute notes to MIDI numbers	148
int	<u>EIGHTH</u>	83
int	<u>EIGHTH QUINTUPLET</u>	84
String	<u>EIGHTH REST STRING</u> The string representing an eighth-rest	89
int	<u>EIGHTH TRIPLET</u>	83
int	<u>EMAJOR</u>	151
int	<u>EN1</u> maps absolute notes to MIDI numbers	118
int	<u>en1</u> maps absolute notes to MIDI numbers	147

int	<u>ES0</u>	maps absolute notes to MIDI numbers	115
int	<u>es0</u>	maps absolute notes to MIDI numbers	145
int	<u>ES1</u>	maps absolute notes to MIDI numbers	113
int	<u>es1</u>	maps absolute notes to MIDI numbers	142
int	<u>ES2</u>	maps absolute notes to MIDI numbers	110
int	<u>es2</u>	maps absolute notes to MIDI numbers	139
int	<u>ES3</u>	maps absolute notes to MIDI numbers	107
int	<u>es3</u>	maps absolute notes to MIDI numbers	136
int	<u>ES4</u>	maps absolute notes to MIDI numbers	105
int	<u>es4</u>	maps absolute notes to MIDI numbers	134
int	<u>ES5</u>	maps absolute notes to MIDI numbers	102
int	<u>es5</u>	maps absolute notes to MIDI numbers	131
int	<u>ES6</u>	maps absolute notes to MIDI numbers	99
int	<u>es6</u>	maps absolute notes to MIDI numbers	128
int	<u>ES7</u>	maps absolute notes to MIDI numbers	96
int	<u>es7</u>	maps absolute notes to MIDI numbers	125
int	<u>ES8</u>	maps absolute notes to MIDI numbers	94
int	<u>es8</u>	maps absolute notes to MIDI numbers	123
int	<u>ES9</u>	maps absolute notes to MIDI numbers	91
int	<u>es9</u>	maps absolute notes to MIDI numbers	120
int	<u>ESN1</u>	maps absolute notes to MIDI numbers	118
int	<u>esn1</u>	maps absolute notes to MIDI numbers	147
int	<u>F0</u>	maps absolute notes to MIDI numbers	115
int	<u>f0</u>	maps absolute notes to MIDI numbers	144

int	<u>F1</u>	maps absolute notes to MIDI numbers	112
int	<u>f1</u>	maps absolute notes to MIDI numbers	142
int	<u>F2</u>	maps absolute notes to MIDI numbers	110
int	<u>f2</u>	maps absolute notes to MIDI numbers	139
int	<u>F3</u>	maps absolute notes to MIDI numbers	107
int	<u>f3</u>	maps absolute notes to MIDI numbers	136
int	<u>F4</u>	maps absolute notes to MIDI numbers	104
int	<u>f4</u>	maps absolute notes to MIDI numbers	133
int	<u>F5</u>	maps absolute notes to MIDI numbers	102
int	<u>f5</u>	maps absolute notes to MIDI numbers	131
int	<u>F6</u>	maps absolute notes to MIDI numbers	99
int	<u>f6</u>	maps absolute notes to MIDI numbers	128
int	<u>F7</u>	maps absolute notes to MIDI numbers	96
int	<u>f7</u>	maps absolute notes to MIDI numbers	125
int	<u>F8</u>	maps absolute notes to MIDI numbers	93
int	<u>f8</u>	maps absolute notes to MIDI numbers	122
int	<u>F9</u>	maps absolute notes to MIDI numbers	91
int	<u>f9</u>	maps absolute notes to MIDI numbers	120
int	<u>FF0</u>	maps absolute notes to MIDI numbers	115
int	<u>ff0</u>	maps absolute notes to MIDI numbers	144
int	<u>FF1</u>	maps absolute notes to MIDI numbers	113
int	<u>ff1</u>	maps absolute notes to MIDI numbers	142
int	<u>FF2</u>	maps absolute notes to MIDI numbers	110
int	<u>ff2</u>	maps absolute notes to MIDI numbers	139

int	<u>FF3</u>	maps absolute notes to MIDI numbers	107
int	<u>ff3</u>	maps absolute notes to MIDI numbers	136
int	<u>FF4</u>	maps absolute notes to MIDI numbers	104
int	<u>ff4</u>	maps absolute notes to MIDI numbers	133
int	<u>FF5</u>	maps absolute notes to MIDI numbers	102
int	<u>ff5</u>	maps absolute notes to MIDI numbers	131
int	<u>FF6</u>	maps absolute notes to MIDI numbers	99
int	<u>ff6</u>	maps absolute notes to MIDI numbers	128
int	<u>FF7</u>	maps absolute notes to MIDI numbers	96
int	<u>ff7</u>	maps absolute notes to MIDI numbers	125
int	<u>FF8</u>	maps absolute notes to MIDI numbers	93
int	<u>ff8</u>	maps absolute notes to MIDI numbers	123
int	<u>FF9</u>	maps absolute notes to MIDI numbers	91
int	<u>ff9</u>	maps absolute notes to MIDI numbers	120
int	<u>FFn1</u>	maps absolute notes to MIDI numbers	118
int	<u>ffn1</u>	maps absolute notes to MIDI numbers	147
String	<u>FIRST SCALE</u>		152
char	<u>FLAT</u>		89
int	<u>FMAJOR</u>		150
int	<u>FN1</u>	maps absolute notes to MIDI numbers	118
int	<u>fn1</u>	maps absolute notes to MIDI numbers	147
int	<u>FOREIGN TONE</u>		86
int	<u>FOUREIGHTIETH</u>		85
int	<u>FS0</u>	maps absolute notes to MIDI numbers	115
int	<u>fs0</u>	maps absolute notes to MIDI numbers	144
int	<u>FS1</u>	maps absolute notes to MIDI numbers	112

int	<u>fs1</u>	maps absolute notes to MIDI numbers	141
int	<u>FS2</u>	maps absolute notes to MIDI numbers	110
int	<u>fs2</u>	maps absolute notes to MIDI numbers	139
int	<u>FS3</u>	maps absolute notes to MIDI numbers	107
int	<u>fs3</u>	maps absolute notes to MIDI numbers	136
int	<u>FS4</u>	maps absolute notes to MIDI numbers	104
int	<u>fs4</u>	maps absolute notes to MIDI numbers	133
int	<u>FS5</u>	maps absolute notes to MIDI numbers	101
int	<u>fs5</u>	maps absolute notes to MIDI numbers	130
int	<u>FS6</u>	maps absolute notes to MIDI numbers	99
int	<u>fs6</u>	maps absolute notes to MIDI numbers	128
int	<u>FS7</u>	maps absolute notes to MIDI numbers	96
int	<u>fs7</u>	maps absolute notes to MIDI numbers	125
int	<u>FS8</u>	maps absolute notes to MIDI numbers	93
int	<u>fs8</u>	maps absolute notes to MIDI numbers	122
int	<u>FS9</u>	maps absolute notes to MIDI numbers	90
int	<u>fs9</u>	maps absolute notes to MIDI numbers	120
int	<u>FSHARP</u>		86
int	<u>FSMAJOR</u>		151
int	<u>FSN1</u>	maps absolute notes to MIDI numbers	118
int	<u>fsn1</u>	maps absolute notes to MIDI numbers	147
int	<u>G0</u>	maps absolute notes to MIDI numbers	115
int	<u>g0</u>	maps absolute notes to MIDI numbers	144
int	<u>G1</u>	maps absolute notes to MIDI numbers	112
int	<u>g1</u>	maps absolute notes to MIDI numbers	141

int	<u>G2</u> maps absolute notes to MIDI numbers	109
int	<u>g2</u> maps absolute notes to MIDI numbers	138
int	<u>G3</u> maps absolute notes to MIDI numbers	107
int	<u>g3</u> maps absolute notes to MIDI numbers	136
int	<u>G4</u> maps absolute notes to MIDI numbers	104
int	<u>g4</u> maps absolute notes to MIDI numbers	133
int	<u>G5</u> maps absolute notes to MIDI numbers	101
int	<u>g5</u> maps absolute notes to MIDI numbers	130
int	<u>G6</u> maps absolute notes to MIDI numbers	98
int	<u>g6</u> maps absolute notes to MIDI numbers	127
int	<u>G7</u> maps absolute notes to MIDI numbers	96
int	<u>g7</u> maps absolute notes to MIDI numbers	125
int	<u>G8</u> maps absolute notes to MIDI numbers	93
int	<u>g8</u> maps absolute notes to MIDI numbers	122
int	<u>G9</u> maps absolute notes to MIDI numbers	90
int	<u>g9</u> maps absolute notes to MIDI numbers	119
int	<u>GBMAJOR</u>	150
int	<u>GF0</u> maps absolute notes to MIDI numbers	115
int	<u>gf0</u> maps absolute notes to MIDI numbers	144
int	<u>GF1</u> maps absolute notes to MIDI numbers	112
int	<u>gf1</u> maps absolute notes to MIDI numbers	141
int	<u>GF2</u> maps absolute notes to MIDI numbers	109
int	<u>gf2</u> maps absolute notes to MIDI numbers	139
int	<u>GF3</u> maps absolute notes to MIDI numbers	107

int	<u>gf3</u>	maps absolute notes to MIDI numbers	136
int	<u>GF4</u>	maps absolute notes to MIDI numbers	104
int	<u>gf4</u>	maps absolute notes to MIDI numbers	133
int	<u>GF5</u>	maps absolute notes to MIDI numbers	101
int	<u>gf5</u>	maps absolute notes to MIDI numbers	130
int	<u>GF6</u>	maps absolute notes to MIDI numbers	99
int	<u>gf6</u>	maps absolute notes to MIDI numbers	128
int	<u>GF7</u>	maps absolute notes to MIDI numbers	96
int	<u>gf7</u>	maps absolute notes to MIDI numbers	125
int	<u>GF8</u>	maps absolute notes to MIDI numbers	93
int	<u>gf8</u>	maps absolute notes to MIDI numbers	122
int	<u>GF9</u>	maps absolute notes to MIDI numbers	90
int	<u>gf9</u>	maps absolute notes to MIDI numbers	119
int	<u>GFN1</u>	maps absolute notes to MIDI numbers	118
int	<u>gfn1</u>	maps absolute notes to MIDI numbers	147
int	<u>GMAJOR</u>		151
int	<u>GN1</u>	maps absolute notes to MIDI numbers	118
int	<u>gn1</u>	maps absolute notes to MIDI numbers	147
int	<u>GS0</u>	maps absolute notes to MIDI numbers	115
int	<u>gs0</u>	maps absolute notes to MIDI numbers	144
int	<u>GS1</u>	maps absolute notes to MIDI numbers	112
int	<u>gs1</u>	maps absolute notes to MIDI numbers	141
int	<u>GS2</u>	maps absolute notes to MIDI numbers	109
int	<u>gs2</u>	maps absolute notes to MIDI numbers	138

int	<u>GS3</u> maps absolute notes to MIDI numbers	106
int	<u>gs3</u> maps absolute notes to MIDI numbers	136
int	<u>GS4</u> maps absolute notes to MIDI numbers	104
int	<u>gs4</u> maps absolute notes to MIDI numbers	133
int	<u>GS5</u> maps absolute notes to MIDI numbers	101
int	<u>gs5</u> maps absolute notes to MIDI numbers	130
int	<u>GS6</u> maps absolute notes to MIDI numbers	98
int	<u>gs6</u> maps absolute notes to MIDI numbers	127
int	<u>GS7</u> maps absolute notes to MIDI numbers	96
int	<u>gs7</u> maps absolute notes to MIDI numbers	125
int	<u>GS8</u> maps absolute notes to MIDI numbers	93
int	<u>gs8</u> maps absolute notes to MIDI numbers	122
int	<u>GSHARP</u>	86
int	<u>GSN1</u> maps absolute notes to MIDI numbers	117
int	<u>gsn1</u> maps absolute notes to MIDI numbers	147
int	<u>HALF</u>	83
int	<u>HALF TRIPLET</u>	83
int	<u>IGNORE</u>	152
int[]	<u>knownNoteValue</u> Note: These must be in ascending order for binary search to work.	85
int[]	<u>knownRestValue</u> Note: These must be in ascending order for binary search to work.	85
int[]	<u>knownTupletValue</u> These are the values that can be used in triplets, either as rests or notes.	85
String	<u>LCROOT</u> Lower case version of pitch class or root c.	89
int	<u>MAX BEAT DENOMINATOR</u>	82
int	<u>MAX BEATS PER BAR</u>	82
int	<u>MAX INTERVAL SIZE</u>	149
int	<u>MAX KEY</u> "maximum" key signature used by Impro-Visor is seven sharps (C#)	88
int	<u>MAX NOTE DURATION</u>	149
int	<u>MAX PITCH</u>	149

int	<u>MAX TS</u>	88
int	<u>MAX VOLUME</u>	82
int	<u>MELODY ONLY</u>	87
int	<u>MIN INTERVAL SIZE</u>	149
int	<u>MIN KEY</u> "minimum" key signature used by Impro-Visor is six flats (Gb)	87
int	<u>MIN NOTE DURATION</u>	149
int	<u>MIN PITCH</u>	149
int	<u>MIN RHYTHM VALUE</u> the minimum allowed rhythm value (not fully implemented)	87
int	<u>MIN TS</u>	88
char	<u>MINUS</u>	90
int	<u>MODA</u>	149
int	<u>MODE</u>	150
int	<u>MODC</u>	149
int	<u>MODD</u>	149
int	<u>MODE</u>	150
int	<u>MODF</u>	149
int	<u>MODG</u>	149
String	<u>NOCHORD</u> The one-and-only symbol for no chord	88
String	<u>NONE</u>	152
int	<u>NUM CHANNELS</u>	82
int	<u>OCTAVE</u> The number of semitones in an octave.	88
int	<u>ONETWENTIETH</u>	85
int	<u>OUT OF BOUNDS</u> the value indicating a variable is out of bounds or invalid	87
int	<u>OVERWRITE</u>	152
char	<u>PLUS</u>	90
int	<u>QUARTER</u>	83
int	<u>QUARTER QUINTUPLET</u>	84
int	<u>QUARTER TRIPLET</u>	83
int	<u>REST</u> the "pitch" of a rest	87
String	<u>REST STRING</u> The string representing a rest	89
int	<u>SAVE</u>	152
int	<u>SEMITONES</u>	148
char	<u>SHARP</u> Input and output symbols used for chords and notes	89
int	<u>SIXTEENTH</u>	83
int	<u>SIXTEENTH QUINTUPLET</u>	84
int	<u>SIXTEENTH TRIPLET</u>	84
int	<u>SIXTIETH</u>	85
char	<u>SLASH</u>	90

String	<u>SLASHSTRING</u> The slash symbol, used for slash-chords and chord repetition	88
String	<u>STYLE_EXTENSION</u>	152
char	<u>T_APPROACH</u>	152
char	<u>T_CHORD</u>	151
char	<u>T_COLOR</u>	151
char	<u>T_NOTE</u>	151
char	<u>T_RANDOM</u>	152
char	<u>T_REST</u>	152
char	<u>T_SCALE</u>	151
int	<u>THIRTYSECOND</u>	83
int	<u>THIRTYSECOND_QUINTUPLET</u>	84
int	<u>THIRTYSECOND_TRIPLET</u>	84
int	<u>TOTALPITCHES</u>	148
int	<u>TWOFORTIETH</u>	85
int	<u>WHOLE</u> Note: All of the following must have no fractional remainder, Hence no SIXTYFOURTH for example.	83

Field Detail

MAX_VOLUME

```
public static final int MAX_VOLUME
```

NUM_CHANNELS

```
public static final int NUM_CHANNELS
```

MAX_BEATS_PER_BAR

```
public static final int MAX_BEATS_PER_BAR
```

DEFAULT_BEATS_PER_BAR

```
public static final int DEFAULT_BEATS_PER_BAR
```

MAX_BEAT_DENOMINATOR

```
public static final int MAX_BEAT_DENOMINATOR
```

DEFAULT_BEAT_DENOMINATOR

```
public static final int DEFAULT_BEAT_DENOMINATOR
```

BEAT

```
public static final int BEAT
```

defines how many slots a BEAT takes up

WHOLE

```
public static final int WHOLE
```

Note: All of the following must have no fractional remainder, Hence no SIXTYFOURTH for example. SIXTYFOURTH_TRIPLET is possible in principle, but we don't have an image for it yet.

HALF

```
public static final int HALF
```

QUARTER

```
public static final int QUARTER
```

EIGHTH

```
public static final int EIGHTH
```

SIXTEENTH

```
public static final int SIXTEENTH
```

THIRTYSECOND

```
public static final int THIRTYSECOND
```

HALF_TRIPLET

```
public static final int HALF_TRIPLET
```

QUARTER_TRIPLET

```
public static final int QUARTER_TRIPLET
```

EIGHTH_TRIPLET

```
public static final int EIGHTH_TRIPLET
```

SIXTEENTH_TRIPLET

```
public static final int SIXTEENTH_TRIPLET
```

THIRTYSECOND_TRIPLET

```
public static final int THIRTYSECOND_TRIPLET
```

QUARTER_QUINTUPLET

```
public static final int QUARTER_QUINTUPLET
```

EIGHTH_QUINTUPLET

```
public static final int EIGHTH_QUINTUPLET
```

SIXTEENTH_QUINTUPLET

```
public static final int SIXTEENTH_QUINTUPLET
```

THIRTYSECOND_QUINTUPLET

```
public static final int THIRTYSECOND_QUINTUPLET
```

DOTTED_HALF

```
public static final int DOTTED_HALF
```

DOTTED_QUARTER

```
public static final int DOTTED_QUARTER
```

DOTTED_EIGHTH

```
public static final int DOTTED_EIGHTH
```

DOTTED_SIXTEENTH

```
public static final int DOTTED_SIXTEENTH
```

FOUREIGHTIETH

```
public static final int FOUREIGHTIETH
```

TWOFORTIETH

```
public static final int TWOFORTIETH
```

ONETWENTIETH

```
public static final int ONETWENTIETH
```

SIXTIETH

```
public static final int SIXTIETH
```

knownNoteValue

```
public static final int[] knownNoteValue
```

Note: These must be in ascending order for binary search to work. What about double dotted?

knownRestValue

```
public static final int[] knownRestValue
```

Note: These must be in ascending order for binary search to work. Rests are not dotted.

knownTupletValue

```
public static final int[] knownTupletValue
```

These are the values that can be used in triplets, either as rests or notes. Note: These must be in ascending order for binary search to work.

DEFAULT_DURATION

```
public static final int DEFAULT_DURATION
```

DEFAULT_SWING

```
public static final double DEFAULT_SWING
```

DEFAULT_METRE

```
public static final int[] DEFAULT_METRE
```

CHORD_TONE

```
public static final int CHORD_TONE
```

Types of notes

COLOR_TONE

```
public static final int COLOR_TONE
```

APPROACH_TONE

```
public static final int APPROACH_TONE
```

FOREIGN_TONE

```
public static final int FOREIGN_TONE
```

CSHARP

```
public static final int CSHARP
```

Enharmonics

DSHARP

```
public static final int DSHARP
```

FSHARP

```
public static final int FSHARP
```

GSHARP

```
public static final int GSHARP
```

ASHARP

```
public static final int ASHARP
```

ALL

```
public static final int ALL
```

Which portions of the leadsheet to export

CHORDS_ONLY

```
public static final int CHORDS_ONLY
```

MELODY_ONLY

```
public static final int MELODY_ONLY
```

MIN_RHYTHM_VALUE

```
public static final int MIN_RHYTHM_VALUE
```

the minimum allowed rhythm value (not fully implemented)

OUT_OF_BOUNDS

```
public static final int OUT_OF_BOUNDS
```

the value indicating a variable is out of bounds or invalid

REST

```
public static final int REST
```

the "pitch" of a rest

blackNotes

```
public static final int[] blackNotes
```

Positions of black notes in the chromatic scale (starting at 0)

MIN_KEY

```
public static final int MIN_KEY
```

"minimum" key signature used by Impro-Visor is six flats (Gb)

MAX_KEY

```
public static final int MAX_KEY
```

"maximum" key signature used by Impro-Visor is seven sharps (C#)

MIN_TS

```
public static final int MIN_TS
```

MAX_TS

```
public static final int MAX_TS
```

OCTAVE

```
public static final int OCTAVE
```

The number of semitones in an octave.

NOCHORD

```
public static final String NOCHORD
```

The one-and-only symbol for no chord

BARSTRING

```
public static final String BARSTRING
```

The bar (measure) symbol

COMMASTRING

```
public static final String COMMASTRING
```

Comma is also usable as a measure symbol

SLASHSTRING

```
public static final String SLASHSTRING
```

The slash symbol, used for slash-chords and chord repetition

CROOT

```
public static final String CROOT
```

The common root for chord table lookup.

LCROOT

```
public static final String LCROOT
```

Lower case version of pitch class or root c.

DEFAULT_DURATION_MSG

```
public static final String DEFAULT_DURATION_MSG
```

The default duration of a note when none is specified.

CMIDI

```
public static final int CMIDI
```

The midi number for middle C

REST_STRING

```
public static final String REST_STRING
```

The string representing a rest

EIGHTH_REST_STRING

```
public static final String EIGHTH_REST_STRING
```

The string representing an eighth-rest

SHARP

```
public static final char SHARP
```

Input and output symbols used for chords and notes

FLAT

```
public static final char FLAT
```

BAR

```
public static final char BAR
```

COMMA

```
public static final char COMMA
```

SLASH

```
public static final char SLASH
```

BACKSLASH

```
public static final char BACKSLASH
```

DOT

```
public static final char DOT
```

PLUS

```
public static final char PLUS
```

MINUS

```
public static final char MINUS
```

G9

```
public static final int G9
```

maps absolute notes to MIDI numbers

GF9

```
public static final int GF9
```

maps absolute notes to MIDI numbers

FS9

```
public static final int FS9
```

maps absolute notes to MIDI numbers

F9

```
public static final int F9
```

maps absolute notes to MIDI numbers

FF9

```
public static final int FF9
```

maps absolute notes to MIDI numbers

ES9

```
public static final int ES9
```

maps absolute notes to MIDI numbers

E9

```
public static final int E9
```

maps absolute notes to MIDI numbers

EF9

```
public static final int EF9
```

maps absolute notes to MIDI numbers

DS9

```
public static final int DS9
```

maps absolute notes to MIDI numbers

D9

```
public static final int D9
```

maps absolute notes to MIDI numbers

DF9

```
public static final int DF9
```


maps absolute notes to MIDI numbers

CS9

```
public static final int CS9
```

maps absolute notes to MIDI numbers

C9

```
public static final int C9
```

maps absolute notes to MIDI numbers

CF9

```
public static final int CF9
```

maps absolute notes to MIDI numbers

BS8

```
public static final int BS8
```

maps absolute notes to MIDI numbers

B8

```
public static final int B8
```

maps absolute notes to MIDI numbers

BF8

```
public static final int BF8
```

maps absolute notes to MIDI numbers

AS8

```
public static final int AS8
```

maps absolute notes to MIDI numbers

A8

```
public static final int A8
```

maps absolute notes to MIDI numbers

AF8

```
public static final int AF8
```

maps absolute notes to MIDI numbers

GS8

```
public static final int GS8
```

maps absolute notes to MIDI numbers

G8

```
public static final int G8
```

maps absolute notes to MIDI numbers

GF8

```
public static final int GF8
```

maps absolute notes to MIDI numbers

FS8

```
public static final int FS8
```

maps absolute notes to MIDI numbers

F8

```
public static final int F8
```

maps absolute notes to MIDI numbers

FF8

```
public static final int FF8
```

maps absolute notes to MIDI numbers

ES8

```
public static final int ES8
```

maps absolute notes to MIDI numbers

E8

```
public static final int E8
```

maps absolute notes to MIDI numbers

EF8

```
public static final int EF8
```

maps absolute notes to MIDI numbers

DS8

```
public static final int DS8
```

maps absolute notes to MIDI numbers

D8

```
public static final int D8
```

maps absolute notes to MIDI numbers

DF8

```
public static final int DF8
```

maps absolute notes to MIDI numbers

CS8

```
public static final int CS8
```

maps absolute notes to MIDI numbers

C8

```
public static final int C8
```

maps absolute notes to MIDI numbers

CF8

```
public static final int CF8
```

maps absolute notes to MIDI numbers

BS7

```
public static final int BS7
```

maps absolute notes to MIDI numbers

B7

```
public static final int B7
```

maps absolute notes to MIDI numbers

BF7

```
public static final int BF7
```

maps absolute notes to MIDI numbers

AS7

```
public static final int AS7
```

maps absolute notes to MIDI numbers

A7

```
public static final int A7
```

maps absolute notes to MIDI numbers

AF7

```
public static final int AF7
```

maps absolute notes to MIDI numbers

GS7

```
public static final int GS7
```

maps absolute notes to MIDI numbers

G7

```
public static final int G7
```

maps absolute notes to MIDI numbers

GF7

```
public static final int GF7
```

maps absolute notes to MIDI numbers

FS7

```
public static final int FS7
```

maps absolute notes to MIDI numbers

F7

```
public static final int F7
```

maps absolute notes to MIDI numbers

FF7

```
public static final int FF7
```

maps absolute notes to MIDI numbers

ES7

```
public static final int ES7
```

maps absolute notes to MIDI numbers

E7

```
public static final int E7
```

maps absolute notes to MIDI numbers

EF7

```
public static final int EF7
```

maps absolute notes to MIDI numbers

DS7

```
public static final int DS7
```

maps absolute notes to MIDI numbers

D7

```
public static final int D7
```

maps absolute notes to MIDI numbers

DF7

```
public static final int DF7
```

maps absolute notes to MIDI numbers

CS7

```
public static final int CS7
```

maps absolute notes to MIDI numbers

C7

```
public static final int C7
```

maps absolute notes to MIDI numbers

CF7

```
public static final int CF7
```

maps absolute notes to MIDI numbers

BS6

```
public static final int BS6
```

GF6

```
public static final int GF6
```

maps absolute notes to MIDI numbers

FS6

```
public static final int FS6
```

maps absolute notes to MIDI numbers

F6

```
public static final int F6
```

maps absolute notes to MIDI numbers

FF6

```
public static final int FF6
```

maps absolute notes to MIDI numbers

ES6

```
public static final int ES6
```

maps absolute notes to MIDI numbers

E6

```
public static final int E6
```

maps absolute notes to MIDI numbers

EF6

```
public static final int EF6
```

maps absolute notes to MIDI numbers

DS6

```
public static final int DS6
```

maps absolute notes to MIDI numbers

maps absolute notes to MIDI numbers

B6

```
public static final int B6
```

maps absolute notes to MIDI numbers

BF6

```
public static final int BF6
```

maps absolute notes to MIDI numbers

AS6

```
public static final int AS6
```

maps absolute notes to MIDI numbers

A6

```
public static final int A6
```

maps absolute notes to MIDI numbers

AF6

```
public static final int AF6
```

maps absolute notes to MIDI numbers

GS6

```
public static final int GS6
```

maps absolute notes to MIDI numbers

G6

```
public static final int G6
```

maps absolute notes to MIDI numbers

maps absolute notes to MIDI numbers

AS5

```
public static final int AS5
```

maps absolute notes to MIDI numbers

A5

```
public static final int A5
```

maps absolute notes to MIDI numbers

AF5

```
public static final int AF5
```

maps absolute notes to MIDI numbers

GS5

```
public static final int GS5
```

maps absolute notes to MIDI numbers

G5

```
public static final int G5
```

maps absolute notes to MIDI numbers

GF5

```
public static final int GF5
```

maps absolute notes to MIDI numbers

FS5

```
public static final int FS5
```

maps absolute notes to MIDI numbers

D6

```
public static final int D6
```

maps absolute notes to MIDI numbers

DF6

```
public static final int DF6
```

maps absolute notes to MIDI numbers

CS6

```
public static final int CS6
```

maps absolute notes to MIDI numbers

C6

```
public static final int C6
```

maps absolute notes to MIDI numbers

CF6

```
public static final int CF6
```

maps absolute notes to MIDI numbers

BS5

```
public static final int BS5
```

maps absolute notes to MIDI numbers

B5

```
public static final int B5
```

maps absolute notes to MIDI numbers

BF5

```
public static final int BF5
```

CS5

```
public static final int CS5
```

maps absolute notes to MIDI numbers

C5

```
public static final int C5
```

maps absolute notes to MIDI numbers

CF5

```
public static final int CF5
```

maps absolute notes to MIDI numbers

BS4

```
public static final int BS4
```

maps absolute notes to MIDI numbers

B4

```
public static final int B4
```

maps absolute notes to MIDI numbers

BF4

```
public static final int BF4
```

maps absolute notes to MIDI numbers

AS4

```
public static final int AS4
```

maps absolute notes to MIDI numbers

A4

```
public static final int A4
```

F5

```
public static final int F5
```

maps absolute notes to MIDI numbers

FF5

```
public static final int FF5
```

maps absolute notes to MIDI numbers

ES5

```
public static final int ES5
```

maps absolute notes to MIDI numbers

E5

```
public static final int E5
```

maps absolute notes to MIDI numbers

EF5

```
public static final int EF5
```

maps absolute notes to MIDI numbers

DS5

```
public static final int DS5
```

maps absolute notes to MIDI numbers

D5

```
public static final int D5
```

maps absolute notes to MIDI numbers

DF5

```
public static final int DF5
```

maps absolute notes to MIDI numbers

ES4

```
public static final int ES4
```

maps absolute notes to MIDI numbers

E4

```
public static final int E4
```

maps absolute notes to MIDI numbers

EF4

```
public static final int EF4
```

maps absolute notes to MIDI numbers

DS4

```
public static final int DS4
```

maps absolute notes to MIDI numbers

D4

```
public static final int D4
```

maps absolute notes to MIDI numbers

DF4

```
public static final int DF4
```

maps absolute notes to MIDI numbers

CS4

```
public static final int CS4
```

maps absolute notes to MIDI numbers

C4

```
public static final int C4
```

maps absolute notes to MIDI numbers

maps absolute notes to MIDI numbers

AF4

`public static final int AF4`

maps absolute notes to MIDI numbers

GS4

`public static final int GS4`

maps absolute notes to MIDI numbers

G4

`public static final int G4`

maps absolute notes to MIDI numbers

GF4

`public static final int GF4`

maps absolute notes to MIDI numbers

FS4

`public static final int FS4`

maps absolute notes to MIDI numbers

F4

`public static final int F4`

maps absolute notes to MIDI numbers

FF4

`public static final int FF4`

maps absolute notes to MIDI numbers

maps absolute notes to MIDI numbers

G3

```
public static final int G3
```

maps absolute notes to MIDI numbers

GF3

```
public static final int GF3
```

maps absolute notes to MIDI numbers

FS3

```
public static final int FS3
```

maps absolute notes to MIDI numbers

F3

```
public static final int F3
```

maps absolute notes to MIDI numbers

FF3

```
public static final int FF3
```

maps absolute notes to MIDI numbers

ES3

```
public static final int ES3
```

maps absolute notes to MIDI numbers

E3

```
public static final int E3
```

maps absolute notes to MIDI numbers

CF4

```
public static final int CF4
```

maps absolute notes to MIDI numbers

BS3

```
public static final int BS3
```

maps absolute notes to MIDI numbers

B3

```
public static final int B3
```

maps absolute notes to MIDI numbers

BF3

```
public static final int BF3
```

maps absolute notes to MIDI numbers

AS3

```
public static final int AS3
```

maps absolute notes to MIDI numbers

A3

```
public static final int A3
```

maps absolute notes to MIDI numbers

AF3

```
public static final int AF3
```

maps absolute notes to MIDI numbers

GS3

```
public static final int GS3
```

B2

```
public static final int B2
```

maps absolute notes to MIDI numbers

BF2

```
public static final int BF2
```

maps absolute notes to MIDI numbers

AS2

```
public static final int AS2
```

maps absolute notes to MIDI numbers

A2

```
public static final int A2
```

maps absolute notes to MIDI numbers

AF2

```
public static final int AF2
```

maps absolute notes to MIDI numbers

GS2

```
public static final int GS2
```

maps absolute notes to MIDI numbers

G2

```
public static final int G2
```

maps absolute notes to MIDI numbers

GF2

```
public static final int GF2
```

EF3

```
public static final int EF3
```

maps absolute notes to MIDI numbers

DS3

```
public static final int DS3
```

maps absolute notes to MIDI numbers

D3

```
public static final int D3
```

maps absolute notes to MIDI numbers

DF3

```
public static final int DF3
```

maps absolute notes to MIDI numbers

CS3

```
public static final int CS3
```

maps absolute notes to MIDI numbers

C3

```
public static final int C3
```

maps absolute notes to MIDI numbers

CF3

```
public static final int CF3
```

maps absolute notes to MIDI numbers

BS2

```
public static final int BS2
```

maps absolute notes to MIDI numbers

D2

```
public static final int D2
```

maps absolute notes to MIDI numbers

DF2

```
public static final int DF2
```

maps absolute notes to MIDI numbers

CS2

```
public static final int CS2
```

maps absolute notes to MIDI numbers

C2

```
public static final int C2
```

maps absolute notes to MIDI numbers

CF2

```
public static final int CF2
```

maps absolute notes to MIDI numbers

BS1

```
public static final int BS1
```

maps absolute notes to MIDI numbers

B1

```
public static final int B1
```

maps absolute notes to MIDI numbers

BF1

```
public static final int BF1
```

maps absolute notes to MIDI numbers

maps absolute notes to MIDI numbers

FS2

public static final int FS2

maps absolute notes to MIDI numbers

F2

public static final int F2

maps absolute notes to MIDI numbers

FF2

public static final int FF2

maps absolute notes to MIDI numbers

ES2

public static final int ES2

maps absolute notes to MIDI numbers

E2

public static final int E2

maps absolute notes to MIDI numbers

EF2

public static final int EF2

maps absolute notes to MIDI numbers

DS2

public static final int DS2

maps absolute notes to MIDI numbers

maps absolute notes to MIDI numbers

FF1

```
public static final int FF1
```

maps absolute notes to MIDI numbers

ES1

```
public static final int ES1
```

maps absolute notes to MIDI numbers

E1

```
public static final int E1
```

maps absolute notes to MIDI numbers

EF1

```
public static final int EF1
```

maps absolute notes to MIDI numbers

DS1

```
public static final int DS1
```

maps absolute notes to MIDI numbers

D1

```
public static final int D1
```

maps absolute notes to MIDI numbers

DF1

```
public static final int DF1
```

maps absolute notes to MIDI numbers

AS1

```
public static final int AS1
```

maps absolute notes to MIDI numbers

A1

```
public static final int A1
```

maps absolute notes to MIDI numbers

AF1

```
public static final int AF1
```

maps absolute notes to MIDI numbers

GS1

```
public static final int GS1
```

maps absolute notes to MIDI numbers

G1

```
public static final int G1
```

maps absolute notes to MIDI numbers

GF1

```
public static final int GF1
```

maps absolute notes to MIDI numbers

FS1

```
public static final int FS1
```

maps absolute notes to MIDI numbers

F1

```
public static final int F1
```

AF0

```
public static final int AF0
```

maps absolute notes to MIDI numbers

GS0

```
public static final int GS0
```

maps absolute notes to MIDI numbers

G0

```
public static final int G0
```

maps absolute notes to MIDI numbers

GF0

```
public static final int GF0
```

maps absolute notes to MIDI numbers

FS0

```
public static final int FS0
```

maps absolute notes to MIDI numbers

F0

```
public static final int F0
```

maps absolute notes to MIDI numbers

FF0

```
public static final int FF0
```

maps absolute notes to MIDI numbers

ES0

```
public static final int ES0
```

CS1

```
public static final int CS1  
  
    maps absolute notes to MIDI numbers
```

C1

```
public static final int C1  
  
    maps absolute notes to MIDI numbers
```

CF1

```
public static final int CF1  
  
    maps absolute notes to MIDI numbers
```

BS0

```
public static final int BS0  
  
    maps absolute notes to MIDI numbers
```

B0

```
public static final int B0  
  
    maps absolute notes to MIDI numbers
```

BF0

```
public static final int BF0  
  
    maps absolute notes to MIDI numbers
```

AS0

```
public static final int AS0  
  
    maps absolute notes to MIDI numbers
```

A0

```
public static final int A0  
  
    maps absolute notes to MIDI numbers
```
